MemScope: A Purpose-Bound Memory Service for Mobile Personal Agents

Anonymous Author(s)[a]

[a]*Institution withheld for review, Anonymous, Anonymous*

Abstract

Persistent memory lets a mobile personal agent maintain preferences, plans, and safety-relevant facts across sessions, but it also converts transient disclosures into a durable attack surface. Existing Android search primitives authorize at package or schema granularity, while contemporary agent-memory systems optimize what to retain and retrieve. Neither boundary expresses that a memory acquired for one purpose may be consumed only by particular apps and tools, until a specified expiry, nor does it conservatively govern summaries derived from multiple sources. We present MemScope, a purpose-bound memory service for mobile personal agents. MemScope places a reference monitor between Android agents and a system-owned persistent store. Every item binds content to provenance, authorized purposes and consumers, app/tool identities, sensitivity, and lifetime; every read carries an authenticated retrieval token derived from the active AppFunction/task context. Authorization precedes semantic ranking, so rejected items do not enter the model context. Derived memories inherit the meet of their cited sources' labels and are admitted only after source and attribution checks. Retrieval-time expiry and background compaction enforce lifetime, while policy-aware eviction separates safety-critical retention from ordinary relevance. Under explicit trust, complete-mediation, and token-authenticity assumptions, we prove admission soundness, monotone restriction of derived labels, returned-plaintext cross-scope non-disclosure, and expiry safety; these properties deliberately do not claim semantic non-inference after an authorized disclosure. We specify a reference implementation and an Android emulator prototype, together with a preregistered evaluation using published memory-extraction and poisoning attacks, long-term-memory benchmarks, and multi-app workloads. We do not claim phone-level energy, thermal, or complete-mediation measurements. Because the project contains no completed measurements, all curves and

result tables are marked as conceptual or pending rather than presented as empirical evidence.

Keywords: mobile personal agents, purpose-bound memory, Android, persistent memory, provenance, reference monitor, AppSearch, label propagation, expiry, policy-aware eviction

---

1. Introduction

A useful personal agent must remember. A calendar correction should survive the conversation in which it was made; a recurring accessibility preference should not need to be repeated; a food allergy mentioned weeks earlier may matter to a shopping or restaurant task. Systems such as MemGPT, MemoryBank, and reflective agents show how persistent stores and generated summaries can extend an agent beyond a finite context window (Packer et al., 2023; Zhong et al., 2024; Shinn et al., 2023). On a phone, however, "memory" is not merely a retrieval-quality component. It is a durable, multi-principal data service positioned among applications, tools, system APIs, cloud models, and one person whose disclosures were made in different contexts.

The security mismatch is concrete. Android AppSearch offers a system-wide index and package/schema visibility controls, and AppFunctions exposes typed callable functionality (Android Developers, 2026b,c,a). These mechanisms identify a caller or function, but they do not natively express that a fact acquired while arranging health care may be used by a medication-reminder tool but not by an advertising or shopping tool; that the authorization expires on a date; or that a summary derived jointly from health and calendar items must retain both restrictions. Conversely, application-level memory libraries normally expose namespaces and retrieval filters chosen by the agent application. A compromised prompt, an over-sharing model, or a malicious tool therefore operates on the same side of the boundary as the data it is supposed to protect.

Recent evidence makes this boundary timely. MEXTRA demonstrates black-box extraction from agent memories (Wang et al., 2025); MINJA injects records through ordinary interaction (Dong et al., 2025); ADAM adapts extraction queries to the apparent memory distribution (Lyu et al., 2026); and SPORE persists reactivation payloads and uses a malicious tool channel to export records despite user-level isolation (Gao et al., 2026b). ToolPrivacyBench separately shows why task success is not a privacy metric:

a tool-using trajectory can complete correctly while sending unnecessary private atoms to intermediate tools (Hu et al., 2026). Memory contamination is durable as well: conversational poisoning and multi-record composition can steer later behavior (Wang et al., 2026; Gao et al., 2026a; Xie et al., 2026). These works do not imply that every memory deployment is exploitable in the same way, but they invalidate the assumption that a user namespace is a sufficient authorization boundary.

1.1. A purpose-bound service boundary

This paper develops MemScope, an Android service that mediates persistent personal-agent memory outside the model. A memory document carries a label containing its provenance, allowed purposes, allowed app/tool consumers, sensitivity class, creation time, and expiry. An agent never selects an arbitrary purpose string and never queries the backing store directly. Instead, a system component issues a signed retrieval token from the active task and AppFunction execution context. The retrieval monitor first constructs an authorized view and only then invokes lexical or vector ranking. Thus an out-of-scope item is not merely down-ranked or hidden by an instruction; its plaintext is absent from the candidate set visible to the ranker and response model.

Derived memories create the less familiar systems problem. An LLM may compress ten messages into a summary, infer a stable preference, or write a reflection used by later tasks. If a new item combines sources, assigning it the least restrictive source label would launder information across contexts. MemScope instead computes the meet of source authorizations: set-valued permissions are intersected, sensitivity is joined upward, and expiry becomes no later than the earliest source expiry. The service requires cited source identifiers and verifies structural attribution before accepting a caller-proposed derivation. A semantic checker may reject unsupported content, but it is not trusted to widen policy. When attribution is uncertain, admission fails closed or the derived item receives a quarantine label.

Lifetime and resource pressure are also authorization concerns. Deleting an index entry in a later maintenance pass is insufficient if an expired item remains readable between expiry and compaction. MemScope checks a trusted monotonic time on every retrieval and performs background deletion as space reclamation rather than as the primary enforcement mechanism. Under storage pressure, policy-aware eviction preserves pinned safety-critical facts when policy permits, while never extending their lifetime or consumer scope.

This differs from treating an LLM context cache as the durable record: recent work shows that safety-critical facts can disappear when compression is evaluated without the future question (Zhu et al., 2026).

1.2. Scope and claims

The formal claims are intentionally narrower than "memory privacy." Given an uncompromised OS service, authentic tokens, complete mediation, correct principal binding, and cryptographic compartment assumptions stated in Section 5, we show that any plaintext item returned by the MemScope API satisfies the declared authorization predicate. We show that a derived label cannot authorize a purpose, consumer, app, tool, or time beyond all its sources. We show that an item whose lifetime has elapsed is not returned, even before physical compaction. These are safety properties of the service interface. They do not prevent an authorized consumer from memorizing or re-transmitting data, do not hide access patterns, and do not provide semantic non-inference over a sequence of individually authorized responses. DP-MemView addresses that complementary cumulative-inference problem under a different interface contract (Kim et al., 2026; Dwork and Roth, 2014).

No completed MemScope measurement dataset was present in the source project. The artifact now includes a typed reference implementation, an emulator prototype plan, and a preregistered evaluation, but no measured outcomes. Accordingly, this manuscript does not claim an achieved extraction rate, latency, energy cost, accuracy, or statistical improvement. It contributes a reproducible system design, formal core, reference implementation, and experimental protocol. Conceptual plots make hypotheses inspectable but are visibly labeled as non-results.

1.3. Contributions

The work makes five contributions directly expanding the source concept:

1. **Labeled mobile-agent memory.** We define a persistent document abstraction whose enforceable dimensions are provenance, purpose, app/tool consumer, sensitivity, and lifetime, coupled to a retrieval-token protocol rooted in Android task and AppFunction context.

2. **Conservative derivation.** We specify source-cited writes, meet-based label propagation, attribution validation, and fail-closed quarantine for summaries and reflections. This prevents the policy widening that would otherwise accompany LLM-based compression.

3. **Lifecycle enforcement under mobile constraints.** We separate retrieval-time expiry from physical deletion and introduce a policy-aware eviction objective that preserves utility and designated safety value without changing authorization.

4. **A precise security argument.** Under stated assumptions, we prove admission soundness, derived-label monotonicity, interface-level cross-scope non-disclosure, expiry safety, and explicit complexity bounds. We also state counterexamples outside those assumptions.

5. **A reproducible evaluation protocol.** We define matched experiments on an Android emulator and a typed reference model against unscoped memory, identity namespaces, AppSearch visibility, prompt-only instructions, and GAAP-style IFC, using memory extraction/poisoning attacks and long-horizon utility workloads. Every presently unmeasured value remains a placeholder.

1.4. Paper organization

Section 2 positions MemScope against memory systems, attacks, information-flow control, and mobile platforms. Sections 3 to 6 define the objects and adversary. Sections 7 and 8 give the architecture and algorithms. Section 9 presents the formal results; Section 10 maps them to a reference implementation and an Android emulator prototype. Section 11 specifies the experiments. Sections 12 and 13 discuss deployment and limits before Section 14 concludes.

2. Related Work

2.1. Long-term memory for language agents

Agent-memory research primarily asks what representation and retrieval policy best sustain behavior over long histories. MemGPT uses virtual-context management and multiple memory tiers (Packer et al., 2023); MemoryBank maintains evolving long-term conversational memory (Zhong et al., 2024); generative agents combine observation, reflection, and planning memories (Park et al., 2023); and Reflexion writes verbal feedback for later trials (Shinn et al., 2023). Retrieval-augmented generation established the broader pattern of retrieving external evidence before generation (Lewis et al., 2020), while ReAct couples model reasoning with tool actions (Yao et al., 2023).

These designs motivate persistent and derived memory but generally place authorization outside the memory abstraction.

LoCoMo and LongMemEval make multi-session recall, temporal reasoning, updates, and abstention measurable (Maharana et al., 2024; Wu et al., 2025). Memora emphasizes invalidated facts and forgetting-aware accuracy (Uddin et al., 2026). These benchmarks are useful for MemScope's utility side, yet authorization requires an additional ground truth: which purpose and consumer may receive each otherwise relevant answer. ProfileFoundry offers structured synthetic people with explicit provenance for constructing such cases without publishing real personal histories (Selvam and Ghosh, 2026). PersonaChat supplies controlled persona conversations but must be augmented with multi-app and policy metadata (Zhang et al., 2018).

Mobile work sharpens the resource problem. A 2026 workshop study compares memory architectures for mobile agents (Zhang et al., 2026), while MOBIMEM supplies profile, experience, and action memories and reports on-device retrieval behavior (Liu et al., 2025). PA-Bench demonstrates that future-blind compression can discard safety-critical facts (Zhu et al., 2026). MemScope is complementary: it does not propose a new semantic memory representation or ranking model. It inserts an authorization and lifecycle boundary before any chosen ranker and constrains derived representations.

2.2. Closest scope and lifecycle mechanisms

Audience-Bound Persistent Memory is the closest lifecycle authorization design. It attaches the recording audience to each item, intersects source audiences for derivations, and authorizes context assembly before model use (Liu, 2026). MemScope adopts the same conservative principle but targets a different principal space and enforcement setting: purpose, source, app, tool, consumer, and lifetime in a system-owned mobile store. The comparison must not be overstated; audience authorization is directly overlapping prior work, not a MemScope invention.

MNC binds declassified inter-agent messages to recipient, purpose, forwarding, lifetime, logging, and memory scopes, then uses a non-LLM monitor for downstream enforcement (Xu et al., 2026). MemScope specializes this idea to persistent mobile-agent memory: token issuance is connected to Android task/AppFunction context; authorization precedes ranking; labels survive derivation and compaction; and implementation cost is measured in an emulator and a typed reference model. Memory as a Service proposes independently callable and governable memory modules (Li, 2025); MemScope

supplies one concrete OS service and policy model rather than a general collaborative-memory agenda.

GAAP tracks private data through agent tools and persistent stores with information-flow control and does not trust the agent to enforce confidentiality (Stanley et al., 2026). MemScope shares the external-monitor principle, but its label dimensions, retrieval view, TTL behavior, and eviction policy are tied to durable on-device memory. SSGM proposes governance controls for evolving memory (Lam et al., 2026); Mi-Memory emphasizes provenance, correction, lifecycle artifacts, and edge/cloud deployment (Liu et al., 2026); and MemPrivacy replaces sensitive spans with local placeholders before cloud-side memory processing (Chen et al., 2026). These are complementary: governance and redaction do not by themselves define per-task authorization over a phone's persistent store.

DP-MemView protects against cumulative attribute inference by controlling response-conditioning views and charging per-attribute privacy ledgers (Kim et al., 2026). MemScope's deterministic label check does not imply differential privacy and cannot stop inference from multiple authorized disclosures. The systems can be composed: MemScope can first restrict the eligible item set, then a DP interface can mediate aggregate transcript leakage. We therefore cite differential privacy only as a boundary and possible extension, not as the proposed method (Dwork and Roth, 2014).

2.3. Memory attacks and purpose-bound tool disclosure

MEXTRA extracts private interactions under black-box access (Wang et al., 2025); ADAM uses adaptive queries (Lyu et al., 2026); and SPORE exploits a malicious tool plus persistent reactivation (Gao et al., 2026b). MINJA injects records through query interactions (Dong et al., 2025). Two MemPoison works expose complementary write-path risks: triggerable conversational poisoning and composition-aware benchmark cases (Wang et al., 2026; Gao et al., 2026a). MemEvoBench studies longer-horizon drift under adversarial memories and noisy tool outputs (Xie et al., 2026). MemScope's read monitor directly targets cross-scope extraction, while provenance and derived-write admission partly address injection. It does not claim to detect arbitrary false but in-scope facts.

General tool-agent security work supplies broader stress tests. Indirect prompt injection shows that untrusted retrieved content can redirect an integrated application (Greshake et al., 2023); AgentDojo offers a dynamic environment for measuring attacks and defenses (Debenedetti et al., 2024); and

ToolEmu uses an emulated sandbox to expose risky tool behavior (Ruan et al., 2023). ToolPrivacyBench is especially relevant because it assigns purpose-specific, need-to-know atoms to tool trajectories (Hu et al., 2026). PiSAs similarly applies contextual integrity to multi-user agent outputs, messages, and memory (Gupta et al., 2026). MemScope converts these contextual expectations into store admission checks for the single-owner, multi-app mobile case.

2.4. Information flow, purpose limitation, and expiry

Security labels have a long formal lineage. Denning models permitted flows with a lattice (Denning, 1976); decentralized labels attach owner-specific policies and safe relabeling rules (Myers and Liskov, 1998, 2000); and language-based IFC distinguishes explicit, implicit, and covert flows (Sabelfeld and Myers, 2003). MemScope uses a small product lattice only for memory authorization. It neither performs whole-program information-flow tracking nor claims noninterference after plaintext crosses the API boundary.

Purpose limitation predates agent memory. Hippocratic databases treat purpose and consent as data-management concerns (Agrawal et al., 2002), and contextual integrity evaluates whether a flow's actors, attributes, and transmission principles fit a social context (Nissenbaum, 2010). Riverbed attaches user policy to data processed by distributed services and isolates compatible policies into universes (Wang et al., 2019). MemScope narrows these ideas to task-scoped retrieval and derived-memory propagation, using OS identities that are available on a phone.

Expiry has both logical and physical meanings. Vanish investigated cryptographic self-destruction after a chosen time (Geambasu et al., 2009). MemScope does not inherit Vanish's stronger claim that all external copies become unreadable. Its guarantee is that the trusted service returns no expired plaintext; encrypted compartment deletion is defense in depth for stored copies.

2.5. Android enforcement points

Android AppSearch offers local and platform storage, ranking, and explicit sharing; PlatformStorage is hosted in the system service, while schema visibility can name authorized packages (Android Developers, 2026b,c). AppFunctions supplies typed function metadata that can ground tool identity and declared task context (Android Developers, 2026a). Hardware-backed Keystore/StrongBox can protect compartment keys from ordinary application

Table 1: Verified closest-work boundary. A check mark records an explicit mechanism, not an overall quality judgment.

System	Persistent store	Purpose	App/tool identity	Derived-label meet	Retrieval-time TTL	Mobile implementation
Audience-Bound Memory (Liu, 2026)	✓	–	–	✓	lifecycle	–
MNC (Xu et al., 2026)	scoped use	✓	recipient/tool scope	message lineage	✓	–
GAAP (Stanley et al., 2026)	✓	permissions	✓	IFC labels	policy dependent	–
MemPrivacy (Chen et al., 2026)	✓	–	edge/cloud boundary	–	–	edge component
Mi-Memory (Liu et al., 2026)	✓	governance	deployment artifacts	provenance	correction/forgetting	edge variant
MemScope (this design)	✓	✓	✓	✓	✓	planned emulator

access (Android Open Source Project, 2026). MemScope extends rather than replaces these mechanisms: per-document purpose/lifetime labels and retrieval tokens are more fine grained than current schema/package visibility.

Prior Android research illustrates both dynamic and static flow tracking. TaintDroid joins multiple runtime tracking granularities across smartphone components (Enck et al., 2010); FlowDroid statically models Android lifecycle and callback behavior (Arzt et al., 2014). MemScope avoids continuous taint tracking inside the model. It enforces complete mediation at a deliberately small store interface, following classic least-privilege and complete-mediation principles (Saltzer and Schroeder, 1975).

2.6. Positioning summary

The novelty claim is therefore bounded: MemScope is a proposed Android enforcement point that combines multi-dimensional per-item scope, authenticated task tokens, derived-item meet propagation, retrieval-time expiry, and mobile resource policies. It is not the first scoped memory proposal, the first information-flow monitor, or the first system to propagate a restrictive label.

3. Preliminaries and notation

3.1. Principals, purposes, and time

Let $\mathcal{A}$ be the finite set of application identities, each bound to a package name and signing certificate; $\mathcal{T}$ the set of tool or AppFunction identities; $\mathcal{P}$ a finite purpose vocabulary; and $\mathcal{C}$ a set of logical consumers such as an assistant profile or system surface. A task context is

$$q = (a_q, t_q, c_q, p_q, \tau_q, n_q), \tag{1}$$

where $a_q \in \mathcal{A}$, $t_q \in \mathcal{T} \cup \{\bot\}$, $c_q \in \mathcal{C}$, $p_q \in \mathcal{P}$, $\tau_q \in \mathbb{T}$ is trusted monotonic time, and n_q is a fresh nonce. The context is carried inside a system-issued token $\kappa_q = \mathsf{Sign}_{K_{\text{iss}}}(q, e_q)$ with token expiry e_q.

Purpose is not inferred from the query string at retrieval time. It is selected from application-authored AppFunction metadata, OS task state, and an optional user-visible purpose confirmation. A classifier may recommend a purpose, but cannot mint a token or expand an item label.

3.2. Memory items and labels

Definition 1 (Labeled memory item)**.** *A memory item is*

$$m_i = (i, x_i, z_i, \ell_i, h_i), \qquad \ell_i = (P_i, A_i, T_i, C_i, s_i, b_i, e_i), \tag{2}$$

where i is an immutable identifier, x_i is encrypted content, z_i is provenance, h_i is an integrity digest, $P_i \subseteq \mathcal{P}$, $A_i \subseteq \mathcal{A}$, $T_i \subseteq \mathcal{T} \cup \{\bot\}$, and $C_i \subseteq \mathcal{C}$ are authorized sets; s_i is sensitivity in a finite ordered set $\mathbb{S}$; and $b_i, e_i \in \mathbb{T}$ are not-before and expiry times with $b_i < e_i$.

Provenance z_i records the source package/certificate, source tool or channel, acquisition token identifier, creation time, cited parent identifiers, transformation type, and content digest. Provenance is immutable except for append-only audit annotations.

Definition 2 (Authorization)**.** *For a verified query context q, the item authorization predicate is*

$$\begin{aligned}\mathsf{Auth}(m_i, q) \iff\ & p_q \in P_i \wedge a_q \in A_i \wedge t_q \in T_i \wedge c_q \in C_i \\ & \wedge\, b_i \le \tau_q < e_i \wedge \tau_q < e_q \wedge \mathsf{ValidProv}(z_i).\end{aligned} \tag{3}$$

Sensitivity is not an independent read grant. It selects encryption, logging, UI, and retention controls after the set-valued authorization test. A deployment may define clearance constraints, but MemScope never treats a high clearance as permission for an unrelated purpose.

3.3. Order and meet

For labels ℓ and ℓ', write $\ell \preceq \ell'$ when ℓ is no more permissive: its allowed sets are subsets, its not-before time is no earlier, its expiry is no later, and its sensitivity is at least as restrictive. For source set S,

$$\mathsf{meet}(S) = \left(\bigcap_{j \in S} P_j, \bigcap_{j \in S} A_j, \bigcap_{j \in S} T_j, \bigcap_{j \in S} C_j, \bigvee_{j \in S} s_j, \max_{j \in S} b_j, \min_{j \in S} e_j \right). \tag{4}$$

An empty allowed-set intersection makes the derivation unusable until an explicit, separately authorized relabeling decision occurs. The ordinary derived-write path cannot widen it.

Table 2: Primary notation.

Symbol	Meaning
$\mathcal{M}$	system-owned set of labeled memory items
m_i, x_i, z_i, ℓ_i	item, encrypted payload, provenance, and label
P_i, A_i, T_i, C_i	allowed purposes, apps, tools, and consumers
s_i, b_i, e_i	sensitivity, not-before time, and expiry
q, κ_q	authenticated task context and signed retrieval token
Auth	per-item authorization predicate
View	authorized pre-ranking candidate view
meet	conservative derived-label meet
$\mathcal{G}$	source/derivation directed acyclic graph
B	storage budget in bytes
R	pluggable lexical/vector/hybrid ranker

3.4. Authorized views and ranking

Given store snapshot $\mathcal{M}_\tau$ and token context q, MemScope constructs

$$\mathsf{View}(\mathcal{M}_\tau, q) = \{m_i \in \mathcal{M}_\tau : \mathsf{Auth}(m_i, q)\}, \tag{5}$$

then evaluates a ranker $R(u_q, \mathsf{View}, k)$ over query text or embedding u_q. This order matters: filtering a model-generated top-k list after ranking can leak through scores, identifiers, timing, or explanations and may allow unauthorized candidates to displace authorized ones.

3.5. Notation table

4. System model

4.1. Deployment

MemScope is a privileged Android service in or adjacent to `system_server`. The service owns the memory database and exposes a Binder API to enrolled agent applications. The backing index reuses AppSearch query and ranking facilities, but item plaintext, label columns, and compartment keys are not directly accessible to callers. Package identity comes from Binder credentials and signing-certificate verification rather than a caller-provided string. Tool identity comes from the AppFunction invocation chain or a system wrapper for non-AppFunction tools.

An agent task begins when a user or system surface invokes an agent with a typed objective. The token issuer maps the task to a purpose in the

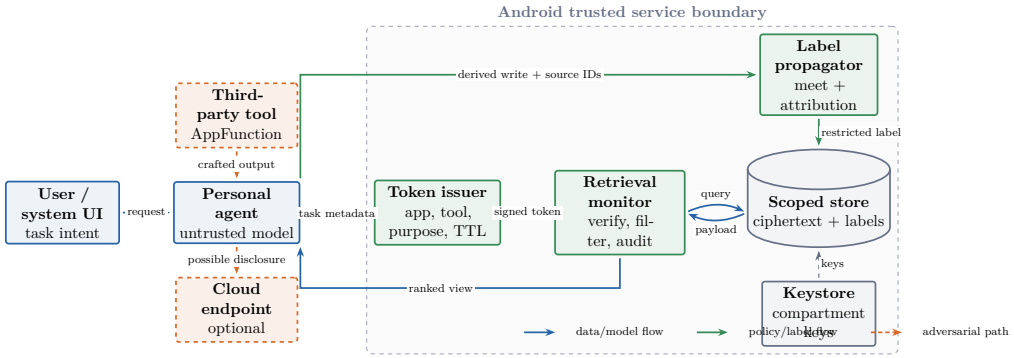


Figure 1: MemScope architecture. The model and tools remain outside the trusted boundary. A task-bound token selects an authorized view before ranking; derived writes use a separate source-cited path. Blue, green, and orange denote ordinary data, trusted enforcement, and adversarially controlled flows, respectively; line style preserves the distinction in grayscale.

installed policy, binds the calling app, consumer, and permitted tools, adds a nonce and short expiry, and signs the context. The agent SDK can request memory with this token but cannot change its fields. The retrieval monitor validates the signature and freshness, filters documents by Equation (3), calls the chosen ranker on Equation (5), decrypts only the returned authorized payloads, and records an audit event.

4.2. Component responsibilities

Scoped Store.. The store maintains payload ciphertext, an indexable authorization projection, provenance, integrity digests, derivation edges, TTL metadata, and an append-only audit trail. Label columns are authenticated as associated data. Compartments can group items with compatible sharing and sensitivity policies so that key deletion provides coarse physical erasure.

Retrieval Monitor.. The monitor is the only plaintext read path. It verifies tokens, constructs authorized views before ranking, enforces result count and query budgets, rate-limits repeated failures, and avoids returning unauthorized identifiers or scores. Audit records refer to opaque item IDs and policy decisions; logging policy prevents the audit stream from becoming a second plaintext store.

Token Issuer.. The issuer binds purposes to declared functions and user-visible tasks. Tokens are single-task or short-lived, carry a nonce, and enumerate the tools in play. Nested tool calls require attenuation: a child token is no

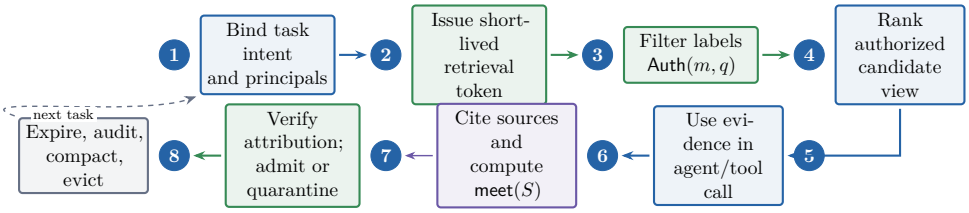


Figure 2: MemScope read–derive–maintain pipeline. Numbering fixes the security-critical order: authorization occurs before ranking, and derivation checks occur before a generated item becomes retrievable.

broader than its parent. Token issuance is a policy decision separate from retrieval relevance.

Label Propagator.. The write path distinguishes direct observations from derivations. Direct items receive defaults from source, task, and data-class policies. Derived items cite existing parent IDs. The propagator checks parent existence and authorization, computes Equation (4), validates the claimed transformation, and rejects caller-proposed labels broader than the computed meet.

Compactor and eviction controller.. The controller removes expired ciphertext, rebuilds indexes, rotates compartment keys, and selects eviction candidates under a byte budget. It can prefer retention of authorized safety-critical items, but it cannot extend expiry or grant access. Tombstones and derivation edges prevent a surviving summary from silently losing its lineage.

4.3. Item lifecycle

An item moves through *proposed*, *admitted*, *quarantined*, *expired*, *tombstoned*, and *physically compacted* states. Only admitted, unexpired items participate in an authorized view. Quarantined items are unavailable to ordinary retrieval; a local review UI may expose them under a distinct administrative purpose. Logical expiry is instantaneous at the API boundary. Physical deletion is asynchronous and auditable.

4.4. Policy representation

Policies are installed manifests signed by the OS, device owner, or enterprise administrator. A rule maps source identities and task metadata to allowed dimensions and lifetime bounds. For example, a calendar availability item may be usable by the scheduling purpose and calendar/messaging tools

for 30 days, while a medical item may be restricted to a health purpose and approved health consumer. Wildcards are explicit and disfavored. Policy compilation rejects contradictory rules and emits a human-readable summary.

4.5. Consistency and concurrency

Retrieval operates on a transactionally consistent store snapshot. Token time is checked both before candidate enumeration and before plaintext release. A concurrent expiry, revocation, or label-narrowing event increments a policy epoch; a query whose epoch changed restarts authorization. Derived writes validate all source versions in one transaction. This prevents time-of-check/time-of-use widening without requiring the ranker itself to be trusted.

5. Threat and privacy model

5.1. Trusted computing base

We trust the kernel mechanisms that deliver Binder identity, the MemScope service, token-signing code and keys, the policy compiler, the AppSearch-derived storage code that MemScope invokes, trusted monotonic time, and hardware-backed key storage when present. We assume verified boot and ordinary Android sandbox guarantees. These assumptions are substantial and must be tested against the actual prototype; the proof does not survive compromise of this trusted computing base.

The model does not trust the language model, its prompt, an agent application's orchestration logic, retrieved untrusted text, third-party tools, or a cloud memory processor. An agent may be confused, prompt-injected, or intentionally request excessive data. A tool may attempt to export arguments or persist instructions. Other apps may invoke exported surfaces or replay old requests. A stolen database image is considered, but an attacker with an unlocked device plus arbitrary code in the trusted service is out of scope.

5.2. Adversary classes

1. A *query adversary* interacts with the assistant and adaptively chooses prompts, following the black-box direction of MEXTRA and ADAM (Wang et al., 2025; Lyu et al., 2026).

2. A *write-path adversary* injects content through normal conversations, notifications, or tool outputs, following MINJA and memory-poisoning settings (Dong et al., 2025; Wang et al., 2026; Gao et al., 2026a).

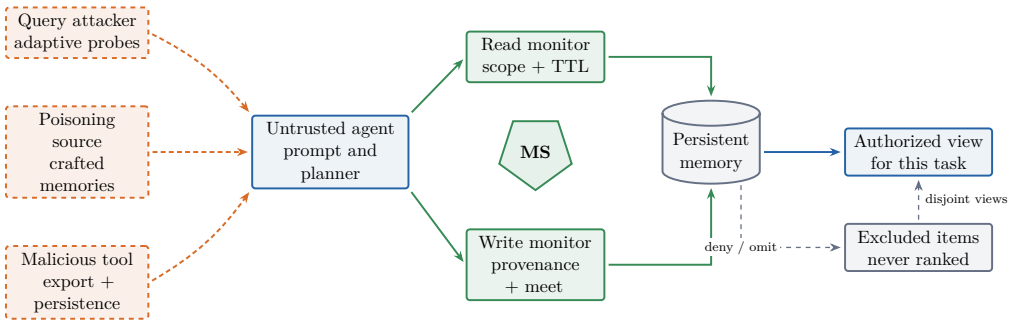


Figure 3: Threat-to-control mapping. Read scoping constrains extraction across purposes and tools; provenance and conservative write admission constrain laundering and some poisoning paths. False but in-scope content and inference from authorized answers remain outside the core guarantee.

3. A *malicious tool* receives calls, returns crafted anchors, and persists reactivation instructions, as in SPORE (Gao et al., 2026b).

4. An *over-sharing agent* completes a valid task but supplies a tool with more private information than the task requires, as measured by ToolPrivacy-Bench (Hu et al., 2026).

5. A *storage adversary* obtains an offline copy of the memory database but not an unlocked OS service or hardware-backed compartment key.

5.3. Assets and security goals

The primary asset is plaintext x_i and its sensitive attributes. Metadata such as item existence, size, access frequency, and timing is a secondary asset but receives only partial protection in this design. The main service goal is:

> For every successful MemScope read, each returned plaintext item is authorized for the authenticated app, tool, consumer, purpose, and time bound to that read; a derived item is no more permissive than every source used to construct it.

Supporting goals are token freshness and attenuation, provenance integrity, retrieval-time expiry, audit completeness, fail-closed label propagation, and confidentiality of offline compartment ciphertext under the platform cryptographic assumptions.

5.4. Explicit non-goals

MemScope does not guarantee that an authorized response reveals no fact inferable about an unauthorized item. It does not revoke information already copied by an authorized consumer, eliminate model-provider logging after authorized cloud disclosure, hide all access patterns, prevent screen capture, protect against kernel/TEE compromise, or stop false in-scope memories. It does not automatically determine the socially correct purpose policy. It does not claim that hardware-backed encryption causes every cached copy to vanish, unlike the stronger historical objective explored by Vanish (Geambasu et al., 2009). Covert channels and timing leakage are measured where feasible but excluded from the core theorem.

5.5. Complete mediation assumption

Every durable personal memory read and write by an enrolled agent must traverse MemScope. This is enforceable only if the evaluation disables or instruments alternative vector stores, caches, logs, and direct cloud-memory APIs. If an application maintains a shadow memory, MemScope cannot govern it. The prototype therefore includes an integration audit and a negative-control build with an intentionally bypassing cache to demonstrate the assumption's necessity.

6. Problem formulation

6.1. Safety and utility objectives

Let Q be a workload of authenticated task contexts and $G(q) \subseteq \mathcal{M}$ the policy ground-truth items entitled and relevant to task q. Let $O(q)$ be the items returned by the store. Authorization violations and misses are

$$V(Q) = \sum_{q \in Q} \sum_{m \in O(q)} \mathbf{1}[\neg \mathsf{Auth}(m, q)], \tag{6}$$

$$L(Q) = \sum_{q \in Q} |G(q) \setminus O(q)|. \tag{7}$$

The safety constraint is $V(Q) = 0$ for service-mediated reads under the model. Utility seeks to reduce $L(Q)$, answer error, latency, and storage subject to safety and resource budgets; energy and thermal costs are outside the measured scope.

Because relevance and authorization are different predicates, maximizing conventional recall over all historically relevant items is incorrect. We report *entitled recall*

$$\mathsf{ER@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|O_k(q) \cap G(q)|}{|G(q)|}, \tag{8}$$

alongside out-of-scope retrieval rate. A method cannot trade a nonzero violation rate for better entitled recall and still satisfy the principal security property.

6.2. Derived-item safety

Let $\mathcal{G} = (\mathcal{I}, E)$ be the derivation DAG, where $(j, i) \in E$ means item i cites item j as a source. For parent set $\mathrm{pa}(i)$, a derived item is policy-valid when

$$\ell_i \preceq \mathsf{meet}(\mathrm{pa}(i)) \quad \wedge \quad \mathsf{Attributed}(x_i, \{x_j : j \in \mathrm{pa}(i)\}). \tag{9}$$

The first conjunct is deterministic. The second is necessarily approximate for natural-language derivations. MemScope treats a false acceptance by the attribution checker as an integrity risk, not as evidence that a broader label is safe. A deployment can require extractive summaries or user confirmation for high-sensitivity classes.

6.3. Storage-budget objective

Let w_i be item size and $y_i \in \{0, 1\}$ indicate retention after compaction. For unexpired, policy-retainable items, define estimated future utility u_i, safety value r_i, sensitivity cost d_i, staleness g_i, and recomputation cost c_i. The planner solves or approximates

$$\max_{\boldsymbol{y}} \quad \sum_i y_i(\alpha u_i + \beta r_i + \gamma c_i - \delta d_i - \eta g_i) \tag{10}$$

$$\text{s.t.} \quad \sum_i y_i w_i \le B, \quad y_i = 0 \text{ if } \tau \ge e_i, \tag{11}$$

with policy-pinned items handled as hard constraints only when their total size fits B. If hard constraints exceed the budget, the service raises an explicit resource fault rather than silently discarding safety-critical records. The objective affects retention, never authorization.

6.4. Design requirements

The formulation yields seven requirements: (R1) authenticated principal and purpose binding; (R2) per-document enforcement before ranking; (R3) conservative derivation; (R4) retrieval-time expiry; (R5) atomic policy/version checks; (R6) auditable lifecycle transitions; and (R7) bounded mobile overhead. The next sections instantiate these requirements.

7. MemScope design

7.1. Task-rooted retrieval tokens

The token issuer observes an OS-mediated task start rather than accepting a purpose asserted by the model. Each installed agent declares AppFunctions, human-readable purposes, tool dependencies, and requested memory classes. At task start, the issuer intersects this declaration with device policy, user/enterprise restrictions, and the caller's package identity. The resulting token contains exactly one purpose, one consumer, one caller app, a bounded tool set, a policy epoch, issuance and expiry times, a nonce, and an optional parent-token digest.

Tokens attenuate. If tool t_1 calls t_2, the child token must satisfy

$$\mathsf{Scope}(\kappa_{t_2}) \preceq \mathsf{Scope}(\kappa_{t_1}) \tag{12}$$

and names t_2 as the active consumer. An agent cannot reuse the parent token as a bearer capability at an unrelated tool because Binder caller identity and the invocation-chain digest are verified together. Replay caches reject nonce reuse for one-shot operations; reusable tokens have short TTLs and per-token query budgets.

Purpose taxonomies inevitably contain ambiguity. MemScope therefore separates three cases. A manifest-matched calendar-creation action maps deterministically to a scheduling purpose. A compound task is decomposed into subtasks with separate tokens. An unknown or unusually sensitive mapping requires user confirmation and yields the narrowest applicable token. A language model may help explain the mapping but is not the policy authority.

7.2. Pre-ranking authorization

AppSearch ranking can combine lexical match, schema filters, properties, and application-defined scores (Android Developers, 2026b). MemScope

adds an indexable authorization projection: hashed IDs for allowed apps, tools, consumers, and purposes, plus not-before/expiry ranges and a policy epoch. The store query includes exact/range constraints that form an over-approximation of Equation (3). The trusted monitor then rechecks each candidate against the authenticated label before decryption and ranking. This two-stage plan reduces work without trusting the index to make the final decision.

Filtering precedes vector ranking and model reranking. Embeddings can be stored within the same compartment or in a separately protected index. If embeddings are treated as sensitive, unauthorized queries do not receive distances or hit counts. The API pads or coarsens certain errors (no results, invalid scope, rate limit) to reduce trivial enumeration, although complete access-pattern confidentiality is outside the formal guarantee.

7.3. Direct writes

A direct write includes payload, source descriptor, active token, requested retention class, and optional user instruction. The service computes policy defaults; the application cannot supply a broader label. Source app and tool are taken from the call chain. Purpose begins at the active task purpose or a policy-defined subset. Consumer sets default to the writing agent and approved system surfaces, never to all installed agents. TTL is capped by the minimum of source policy, task policy, and user choice. High-sensitivity data can require a separate compartment and suppress content-bearing audit fields.

User corrections are versioned rather than overwritten invisibly. A correction creates a new item linked by `supersedes`; the old item becomes invalid for ordinary retrieval at the commit epoch and is later compacted. This supports Memora-style update evaluation (Uddin et al., 2026) and makes stale-memory reuse observable.

7.4. Derived writes and attribution

Summaries, extracted preferences, and reflections use a separate call. The proposal contains generated payload x_d, transformation type, ordered parent IDs S, and optional spans mapping claims to sources. MemScope performs four checks:

1. Every parent exists, is unexpired, and was visible to the token used for the derivation.

2. Parent versions and digests match the snapshot on which generation occurred.

3. The proposed label is equal to or more restrictive than $\mathsf{meet}(S)$.

4. An attribution verifier finds no material claim unsupported by the cited sources under the configured verifier policy.

The deterministic first three checks establish policy monotonicity. The fourth is a content-integrity heuristic whose errors are measured. For safety-critical or highly sensitive items, the allowed modes can be restricted to extractive selection, structured transforms, or user-confirmed abstraction. An abstractive checker cannot authorize policy widening; a pass merely allows storage under the already conservative meet.

If the parent intersection is empty, the service does not guess. It may store the item in a quarantined administrative compartment for review, or require separate scoped derivatives. If a parent expires, a derived item already has an expiry no later than that parent and therefore expires too. A direct user grant can create a new declassification event, but that operation is outside ordinary derivation, requires object-specific confirmation, and is recorded as a new provenance edge.

7.5. Expiry, deletion, and revocation

Read-time TTL checks are authoritative. The compactor's role is to reclaim space and reduce residual ciphertext, not to decide whether an item is logically accessible. On expiry, the monitor returns neither payload nor content-derived score. The compactor deletes the item, removes index entries, and updates derivation tombstones. If a whole compartment is expired or revoked, destroying its wrapped key accelerates physical erasure.

Revocation differs from expiry because it may occur before e_i. Policy epochs make revocation visible to active queries: label narrowing or deletion increments the relevant epoch, and release revalidates it. An audit record contains the rule and version responsible for the decision. Long-running agent tasks must refresh tokens after their expiry; refresh re-evaluates policy rather than extending the old grant automatically.

7.6. Policy-aware eviction

When storage exceeds its soft budget, expired and invalidated items leave first. Remaining candidates are grouped by compartment and derivation

dependencies. Evicting a parent whose derived item still exists is allowed only if the derivation record retains a digest and audit metadata sufficient for lineage; deployments requiring re-verification pin the parent instead. The objective in Equation (11) uses predicted entitled reuse, safety class, sensitivity cost, staleness, and recomputation cost. All learned scores are advisory. Hard policy constraints—for example, retaining a confirmed allergy until a user-specified date—are explicit and size-accounted.

This policy avoids a subtle conflation. Sensitivity can argue for earlier deletion because storage risk is high; safety criticality can argue for retention because forgetting is harmful. MemScope therefore represents both axes rather than treating "sensitive" as automatically retained or evicted. If the budget cannot satisfy hard obligations, the UI reports the conflict and requests a storage/policy decision.

7.7. Auditing and user controls

The audit log records token ID, caller certificate digest, purpose, tool, opaque item IDs, decision codes, bytes returned, policy version, and timing. Plaintext query and item content are excluded by default. A user-facing view answers: what was remembered, from which source, why it was used, which tool received it, when it expires, and what derived items depend on it. The user can narrow scope, expire an item, correct content, or revoke a tool. Widening requires a distinct confirmation because it changes the information-flow policy.

7.8. Failure behavior

MemScope fails closed on invalid signatures, unknown identities, ambiguous tool chains, stale epochs, malformed labels, empty required intersections, unavailable key material, and attribution-checker failures. Fail-closed behavior can reduce utility, so the system distinguishes policy denial from transient infrastructure failure in local diagnostics while presenting a non-revealing error to untrusted callers. The evaluation measures both denied legitimate requests and attempted violations.

7.9. Why an OS service

Application libraries cannot reliably mediate competing apps and tool providers, and prompt instructions place enforcement in the component exposed to prompt injection. An OS service already sees package identities, signing certificates, AppFunction metadata, lifecycle events, storage budgets,

Algorithm 1 MemScope retrieval

Require: signed token κ, query u, result limit k

$q \leftarrow \mathsf{VerifyToken}(\kappa, \mathsf{BinderCaller}, \mathsf{ToolChain})$
if $q = \bot$ or $\mathsf{Now}() \geq q.e_q$ **then**
    return DENY
end if
$v \leftarrow \mathsf{PolicyEpoch}(q)$
$C \leftarrow \mathsf{IndexCandidates}(u, \mathsf{Projection}(q))$
$A \leftarrow [\,]$
for all $m \in C$ **do**
    if $\mathsf{IntegrityOK}(m)$ and $\mathsf{Auth}(m, q)$ **then**
        append m to A
    end if
end for
$O \leftarrow \mathsf{Rank}(u, A, k)$
if $v \neq \mathsf{PolicyEpoch}(q)$ or $\mathsf{Now}() \geq q.e_q$ **then**
    return RETRIEVE(κ, u, k) ▷ bounded restart
end if
$X \leftarrow \mathsf{DecryptAuthorized}(O, q)$
$\mathsf{Audit}(q, O, \mathsf{ALLOW})$
return X

and hardware-key APIs. AppSearch's package/schema visibility demonstrates a compatible system-service pattern (Android Developers, 2026c); MemScope adds per-document task semantics and derivation lineage rather than discarding the existing index.

8. Algorithms

Algorithm 1 performs a redundant authorization check after index filtering because index projection is an optimization, not the trusted policy decision. Production code bounds restarts and returns a transient error during pathological policy churn.

The greedy compactor is a documented approximation to Equation (11), not an optimality claim. An exact knapsack solver can be used offline for small stores as an oracle in evaluation.

9. Security and performance analysis

9.1. Assumptions

Assumption 1 (Identity and token authenticity)**.** *Binder caller identity, package signatures, and the AppFunction/tool chain are correctly reported. The issuer's signature is existentially unforgeable for the adversary, nonces/expiries are checked, and token fields cannot be altered without detection.*

Assumption 2 (Complete mediation)**.** *All persistent personal-memory reads and writes in the evaluated system traverse MemScope; callers cannot directly access plaintext, compartment keys, or an equivalent shadow store.*

Assumption 3 (State integrity and time)**.** *Labels, provenance, derivation edges, and policy epochs are integrity-protected; transactions provide the snapshot/version behavior described in Section 4; and* $\mathsf{Now}()$ *is a trusted monotonic clock.*

Assumption 4 (Ranker confinement)**.** *The ranker receives only the candidate view provided by MemScope and has no independent plaintext or decryption path. Its output is a subset of its input item identifiers.*

These assumptions are test obligations, not environmental facts. Section 13 gives failure cases when they do not hold.

Lemma 1 (Meet monotonicity)**.** *For every nonempty parent set* S *and each* $j \in S$, $\mathsf{meet}(S) \preceq \ell_j$.

Proof. Set intersection is a subset of every operand. The maximum not-before time is no earlier than each operand's time; the minimum expiry is no later; and the sensitivity join is at least as restrictive as every operand. These are exactly the coordinates of $\preceq$. □

Lemma 2 (Authorized-view invariant)**.** *Under Assumptions 1–3, every* $m \in \mathsf{View}(\mathcal{M}_\tau, q)$ *satisfies* $\mathsf{Auth}(m, q)$ *at the snapshot used for release.*

Proof. The result follows from the set comprehension in Equation (5). Operationally, Algorithm 1 verifies q, applies the predicate to every candidate, and restarts if the policy epoch changes before release. Index projection may add false candidates but cannot bypass the trusted recheck. □

Theorem 1 (Retrieval admission soundness)**.** *Under Assumptions 1–4, if Algorithm 1 returns plaintext for item m_i in response to token κ_q, then* $\mathsf{Auth}(m_i, q)$ *holds at the release snapshot.*

Proof. See Section Appendix A.2. The proof composes token verification, Lemma 2, ranker confinement, epoch revalidation, and authorized-only decryption. □

Theorem 2 (Derived-label safety)**.** *Suppose Algorithm 2 admits derived item d with nonempty parent set S. Then $\ell_d \preceq \ell_j$ for every $j \in S$. Therefore, authorization of d cannot arise from a purpose, app, tool, consumer, or time interval absent from any one parent.*

Proof. Algorithm 2 admits only if $\ell_d \preceq \mathsf{meet}(S)$. Lemma 1 gives $\mathsf{meet}(S) \preceq \ell_j$ for each parent. Transitivity yields $\ell_d \preceq \ell_j$. □

Corollary 1 (Expiry inheritance)**.** *An admitted derived item's expiry is no later than the expiry of every parent, unless a separate, explicitly authorized declassification creates a new item outside Algorithm 2.*

Theorem 3 (Returned-plaintext cross-scope non-disclosure)**.** *Let contexts q_1 and q_2 differ in at least one authorization dimension. Under Assumptions 1–4, an item m with* $\mathsf{Auth}(m, q_1)$ *and* $\neg\mathsf{Auth}(m, q_2)$ *is not returned as plaintext by MemScope under q_2, including through any chain of ordinary admitted derivations.*

Proof. See Section Appendix A.4. The direct case follows from Theorem 1; the derived case uses Theorem 2 inductively on the acyclic provenance graph. □

Theorem 4 (Retrieval-time expiry safety)**.** *Under Assumptions 1–3, if trusted time $\tau \geq e_i$, MemScope does not return plaintext x_i under any ordinary retrieval token, regardless of whether physical compaction has deleted the ciphertext.*

Proof. At $\tau \geq e_i$, the strict inequality $\tau < e_i$ in Equation (3) is false. By Theorem 1, returned plaintext must satisfy authorization. Hence x_i cannot be returned. Physical deletion is not used in the argument. □

Proposition 1 (Retrieval complexity)**.** *Let n be the total number of items, c the number emitted by the indexed projection, d embedding dimension, and k requested results. Given an index lookup cost $I(n)$, authorization rechecking*

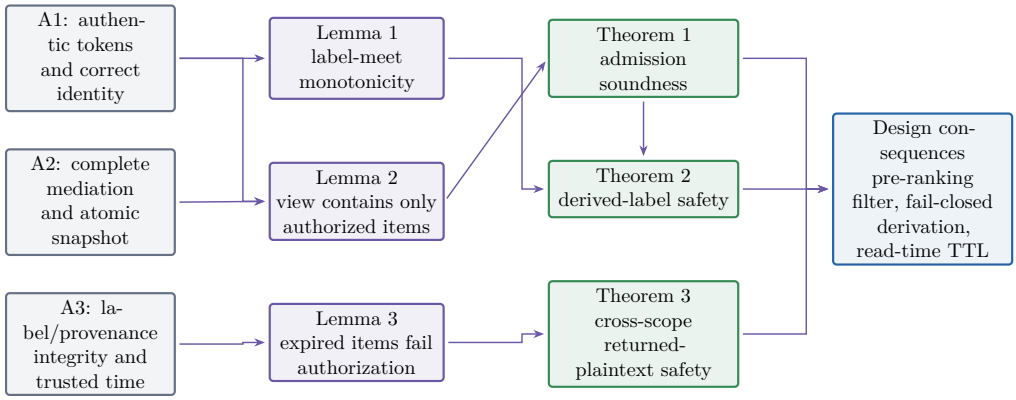


Figure 4: Proof dependency map. The routed edges emphasize that interface guarantees depend on explicit trusted-system assumptions; they are not claims about arbitrary model behavior.

is $O(c)$, exact dense scoring is $O(cd)$, and selecting the top k is $O(c \log k)$. Label storage is $O(n(\bar{p} + \bar{a} + \bar{t} + \bar{c}))$ identifiers before compression, where the bars are mean allowed-set sizes.

Proof. Each candidate has a constant number of hashed set-membership and range checks under indexed/hashed label representations. Dense dot products cost $O(d)$ per authorized candidate, and a size-k heap costs $O(\log k)$ per insertion. The storage bound sums the label-set cardinalities. □

Remark 1 (No semantic non-inference). Theorem 3 is an API trace property about returned stored plaintext. It does not say that answers generated from authorized items reveal zero information about an unauthorized attribute. Repeated authorized views may require a mechanism such as DP-MemView (Kim et al., 2026).

Remark 2 (Attribution checker). Theorem 2 is about labels, not semantic faithfulness. If a generated summary invents a false statement, a restrictive label does not make the content true. Attribution precision and recall are empirical quantities in Section 11.

10. Reference implementation and Android prototype plan

10.1. Two implementation layers

The artifact has two layers. A typed Python reference model implements the token, policy, retrieval, derivation, compaction, and metric contracts and

is the primary vehicle for matched experiments. An Android application prototype running on the standard emulator implements the same API surface with an AIDL Binder service, service-owned encrypted storage, software-backed Android Keystore keys, audit events, and TTL enforcement. Neither layer patches `system_server` or establishes complete mediation on a commercial device; a system-level AppSearch/AppFunctions variant is future work.

10.2. Service and API surface

The prototype exposes a Binder interface with four principal calls: `issueTaskToken`, `queryMemory`, `putObservation`, and `putDerivation`. Maintenance calls cover correction, revocation, compaction, export, and policy inspection. Public SDK objects contain opaque IDs and structured explanations; they never expose raw database handles.

The emulator prototype keeps authorization fields in service-owned tables and restricts the database to the service process. Queries pass through the service, which filters before ranking. A system-level variant would move projection filtering into AppSearch PlatformStorage; comparing the two is left to future work.

10.3. Schema

The document schema contains: item ID; ciphertext blob or encrypted file reference; embedding reference; source package certificate digest; source tool/function ID; acquisition token digest; purpose/app/tool/consumer ID arrays; sensitivity; not-before and expiry; policy epoch; parent IDs and digests; transformation type; state; size; and compact audit counters. Set IDs are device-local keyed hashes to limit disclosure from a raw index image. Integrity tags cover the label, provenance, and ciphertext reference.

Payload encryption uses per-compartment data-encryption keys wrapped by Android Keystore. The emulator supplies software-backed keys; StrongBox is future work (Android Open Source Project, 2026). The design does not assume that a secure element makes an unlocked authorized read safe from a compromised system service. It protects keys from ordinary application access and raises the cost of offline database theft, but we do not claim vendor secure-hardware behavior.

10.4. Token binding

AppFunction metadata provides the declared function and schema (Android Developers, 2026a). A system interceptor records the calling UID,

package certificate, target provider, function identifier, and parent task. The purpose registry maps this tuple to a stable purpose ID. For legacy tools, a mediated adapter supplies equivalent identifiers; an unmediated network endpoint cannot receive a MemScope token.

Tokens use a compact canonical encoding, hardware- or OS-keystore-backed signatures, a 128-bit nonce, and a policy epoch. Verification caches may store positive signature results until token expiry but still check caller/tool binding on every request. Tests cover field mutation, replay, confused-deputy chains, clock rollback, stale epochs, and cross-user Android profiles.

10.5. Label and attribution engine

The deterministic propagator is ordinary systems code. The optional semantic verifier is replaceable: lexical coverage for extractive transforms; natural-language inference for abstractive summaries; and a small on-device model for offline operation. Candidate models include the small on-device class named in the source concept. The verifier produces supported/unsupported/uncertain plus cited spans; uncertainty quarantines high-sensitivity items. Model output cannot directly set label fields.

The experimental build logs verifier decisions and a content-free feature vector so that false acceptance and rejection can be audited against human labels. No test split is used to tune production thresholds after reporting begins.

10.6. Concurrency and recovery

SQLite/AppSearch transactions associate a snapshot version with parent reads and derived writes. A write-ahead journal records intent before key and index changes. Recovery replays only operations whose integrity and policy epoch remain valid. Token replay state and recent audit roots persist across reboot. Compaction is idempotent: tombstoning, index deletion, ciphertext deletion, and key cleanup can be retried without restoring logical accessibility.

10.7. Instrumentation

Latency is divided into Binder, signature, projection, policy recheck, ranking, decryption, and audit components on the emulator. Storage accounting separates payload, embeddings, labels, provenance DAG, audit, and index. The Python reference model supplies scale measurements at 10^2–10^5 items. Energy, thermal, and CPU/NPU counters on physical phones are not measured and are not claimed.

10.8. Compatibility

An Agent SDK adapter converts common memory operations—add, search, update, delete—into the MemScope API. Mem0/Letta-style identity namespaces remain a baseline but are not treated as equivalent authorization. Applications unwilling to integrate remain outside complete mediation and cannot claim MemScope protection. The system can expose migration tooling that imports records under a conservative quarantine policy until provenance and scope are assigned.

11. Evaluation methodology and pending results

Status. Placeholder / expected trend only; replace with actual experimental results before submission. The artifact contains a typed reference implementation and an Android emulator prototype plan, but no completed measurements. This section is a preregistered protocol: it states questions, factors, metrics, controls, and analysis before final measurements exist. The numbers in Figure 6 are illustrative coordinates only.

11.1. Research questions

RQ1: Enforcement.
Does MemScope reduce out-of-scope plaintext retrieval and attack extraction yield relative to unscoped memory, namespace scoping, AppSearch visibility, and prompt-only instructions?

RQ2: Utility.
What entitled recall and answer accuracy are lost as purpose/app/tool scopes narrow, and where is the privacy–utility Pareto frontier?

RQ3: Derivation.
How accurately does source attribution accept faithful summaries and reject unsupported or policy-laundering derivations?

RQ4: Lifecycle.
Are expiry, correction, revocation, and invalidation respected across active queries, reboot, compaction delay, and derived-item chains?

RQ5: System cost.
What latency, storage, and compaction overhead arises in the emulator prototype and the reference model, and which component dominates?

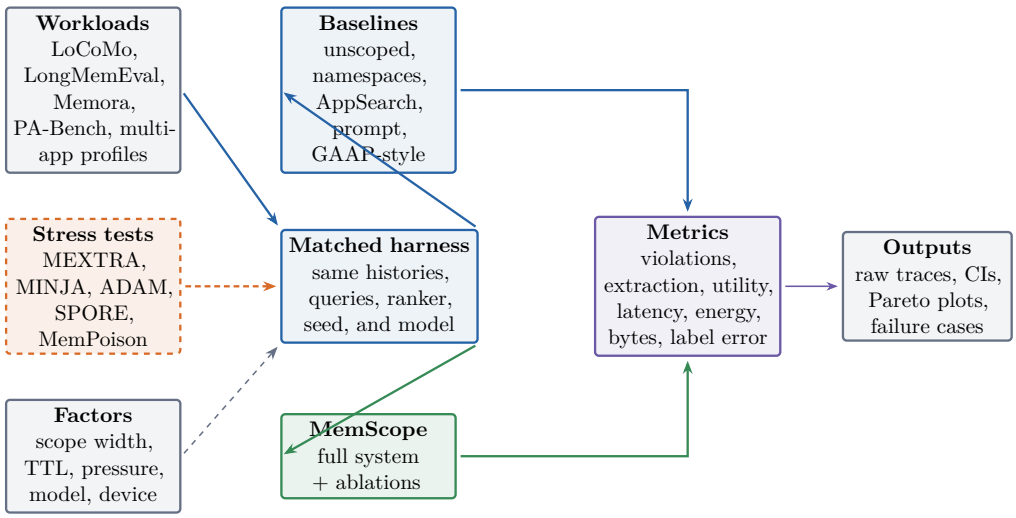


Figure 5: Preregistered experimental pipeline. Baselines and MemScope receive identical workload and model inputs; only the memory boundary changes. No box in this figure denotes a completed result.

RQ6: Retention.
Does policy-aware eviction preserve entitled utility and designated safety-critical facts better than recency, relevance, or size-only eviction under the same byte budget?

11.2. Evaluation environment

The primary environment is a standard Android emulator (API 34 or 35) running the prototype, plus the typed Python reference model for matched algorithmic experiments. A physical-device track is out of scope: the paper does not report phone energy, thermal, StrongBox, or Knox measurements. Every run records host CPU and RAM, emulator image and build fingerprint, Android API level, Python version, and model identifiers. Exact model identifiers, dates, decoding settings, prompts, and library versions are frozen. A model update creates a new experimental stratum. The ranking engine, embeddings, and k are held constant across matched memory-boundary comparisons.

11.3. Workloads and datasets

Long-horizon utility.. LoCoMo supplies very long conversational histories and question answering (Maharana et al., 2024); LongMemEval tests extraction,

multi-session and temporal reasoning, updates, and abstention (Wu et al., 2025); Memora adds invalidated memories and forgetting-aware accuracy (Uddin et al., 2026). We attach policy labels to each evidence item using a documented annotation protocol. Original benchmark scores remain separately reported so the augmentation does not masquerade as an official task.

Safety-critical retention.. The PA-Bench protocol hides future questions during compression and tests whether facts such as allergies survive (Zhu et al., 2026). We reproduce the allowed public protocol and add purpose/lifetime annotations. The central comparison holds the byte budget fixed across eviction methods.

Multi-app mobile workload.. Synthetic people from ProfileFoundry seed consistent calendar, messaging, shopping, travel, and health histories (Selvam and Ghosh, 2026). PersonaChat-style dialogue realizes natural interactions (Zhang et al., 2018). Each episode has a policy oracle listing entitled consumers for every atom. The workload includes cross-app joins that are authorized, joins requiring separate tokens, updates, revocations, ambiguous purposes, and malicious tool outputs. AndroidWorld can host GUI tasks where license and version compatibility permit (Rawles et al., 2024).

11.4. Attacks and stress tests

MEXTRA evaluates black-box extraction (Wang et al., 2025); ADAM evaluates adaptive extraction (Lyu et al., 2026); SPORE evaluates a malicious tool and persistent reactivation (Gao et al., 2026b); MINJA evaluates query-only memory injection (Dong et al., 2025); and the two MemPoison formulations evaluate triggerable and compositional corruption (Wang et al., 2026; Gao et al., 2026a). Each attack is run from the authors' released artifact when available and version-pinned. If no artifact exists, a reimplementation is labeled as such and validated on the paper's reported setup before being used against MemScope.

Attack success is stratified into: direct unauthorized item return; model paraphrase attributable to an unauthorized item; unauthorized tool argument; poisoned item admission; poisoned behavior activation; and availability degradation. The formal theorem covers only the first category at the service API. Reporting the categories separately prevents a narrow systems property from being advertised as complete attack prevention.

11.5. Baselines

1. **Unscoped:** one user-level memory with the same ranker and byte budget.
2. **Namespace:** Mem0/Letta-style user/agent/run identifiers, with no purpose or tool label.
3. **AppSearch visibility (emulated):** package/schema sharing using the documented visibility APIs and caller identity rules (Android Developers, 2026c). The baseline runs inside the harness; it does not modify a platform AppSearch module.
4. **Prompt-only:** the model is instructed to respect the same textual policy, but the store returns ordinary top-k candidates.
5. **GAAP-style IFC:** an execution monitor tracks private data through tools and persistent state, configured as closely as accessible artifacts allow (Stanley et al., 2026). Any deviation from the published system is documented.
6. **Audience-bound/scoped concurrent work:** executable artifacts for Audience-Bound Memory or MNC are included when compatible; otherwise comparison remains analytical, not numerical (Liu, 2026; Xu et al., 2026).

All baselines use identical histories, queries, ranker, model calls, seeds, device state, and budgets. If a baseline cannot represent a policy, the experiment reports "unsupported" rather than silently approximating it as a denial or success.

11.6. Factors and ablations

The factorial core varies: number of items (10^2–10^5 where feasible); scope width; number of apps/tools; TTL distribution; sensitivity and safety mix; write/read ratio; memory pressure; attack budget; ranking method; local verifier model; and network availability. Ablations remove one label dimension at a time, pre-ranking filtering, token/tool binding, derived-label meet, attribution verification, read-time TTL, epoch revalidation, policy-aware eviction, and compartment encryption.

The critical negative controls intentionally violate assumptions: a shadow cache bypasses MemScope; a stale token is replayed; the attribution checker is replaced with always-accept; and an authorized tool copies plaintext after receipt. These controls should defeat the corresponding guarantee and make its boundary visible.

Table 3: **Placeholder / expected trend only; replace with actual experimental results before submission.** No numeric outcomes are populated.

Method	Out-of-scope retrieval	MEXTRA yield	SPORE yield	MINJA admission	Entitled accuracy	p95 latency
Unscoped	pending	pending	pending	pending	pending	pending
Namespace	pending	pending	pending	pending	pending	pending
AppSearch visibility	pending	pending	pending	pending	pending	pending
Prompt-only	pending	pending	pending	pending	pending	pending
GAAP-style	pending	pending	pending	pending	pending	pending
MemScope	pending	pending	pending	pending	pending	pending

11.7. Metrics

Security metrics are out-of-scope retrieval rate, unique-record extraction yield, sensitive-atom disclosure, poisoned-write acceptance, trigger activation, and queries/tool calls to success. Utility metrics are entitled Recall@k, answer exact/F1 or benchmark-specific accuracy, FAMA where applicable, task completion, abstention calibration, and safety-critical fact survival. Systems metrics are median and tail latency, CPU time, bytes per item, index amplification, compaction pause, and key-operation cost. Energy, thermal, and CPU/NPU counters on physical phones are out of scope. Label metrics are per-dimension precision/recall, exact-label match, false widening, false narrowing, attribution false acceptance/rejection, and quarantine rate.

11.8. Statistical plan

Primary comparisons are paired by workload instance and random seed. Continuous metrics report medians, bootstrap 95% confidence intervals, and full distributions; bounded success rates report Wilson or exact binomial intervals. Multiple attack/baseline comparisons use a declared family and Holm correction. Effect sizes accompany p-values. Environment (reference model versus emulator) is treated as a block. Runs with emulator slowdown are not discarded silently: the preregistered rule either reruns after host cooldown or reports a separate stratum.

At least three independent process/device restarts are used for microbenchmarks and enough episode repetitions are chosen from a prospective power analysis based on pilot variance. Pilots are not mixed with final data. All exclusions and failed runs appear in the artifact log.

11.9. Pending result tables

11.10. Reproducibility checklist

Before submission the artifact must include reference-implementation and emulator commits; build manifests; host and emulator image fingerprints;

Table 4: **Placeholder / expected trend only; replace with actual experimental results before submission.** Planned system-cost decomposition for the reference model and the Android emulator. No physical-device energy or thermal measurements are planned.

Component	Reference model	Emulator
Token verification	pending	pending
Label projection	pending	pending
Policy recheck	pending	pending
Ranking	pending	pending
Decryption/audit	pending	pending
Storage per item	pending	pending

policy schemas and compiled examples; benchmark licenses and preparation hashes; attack artifact versions; model identifiers and prompts; seed lists; raw JSONL traces; analysis scripts; environment lockfiles; figure-generation scripts; host/emulator metadata; and a one-command smoke test. Claims in the final abstract and conclusion must be regenerated from machine-readable result summaries, not manually copied from preliminary notebooks.

12. Discussion

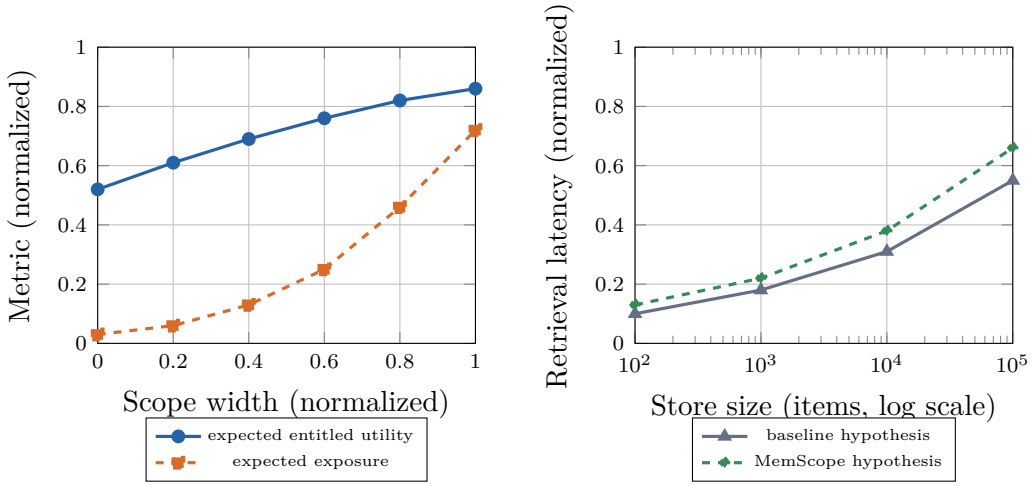
12.1. Policy provenance is as important as data provenance

A label can be internally consistent yet socially wrong. MemScope therefore records which manifest, user choice, enterprise rule, and OS default produced each dimension. Review interfaces should display this policy provenance next to data provenance. When policy changes, the system can identify affected items and derived descendants rather than relabeling an undifferentiated store.

Contextual integrity is useful here because it frames privacy as appropriate flow among actors, attributes, and transmission principles, not simply secrecy (Nissenbaum, 2010). MemScope implements only a machine-checkable approximation. User studies are needed to learn whether purpose names and tool identities are understandable, whether defaults match expectations, and how often prompts for ambiguous compound tasks create fatigue.

12.2. Interaction with information-flow control

MemScope protects admission into model context. A whole-agent IFC system can continue tracking data after admission and constrain tool output,



(a) Hypothesized utility–exposure trade-off.

(b) Hypothesized authorization overhead.

Figure 6: **Conceptual expected trends, not experimental results.** Coordinates are illustrative only and must be replaced by measurements and uncertainty intervals before submission.

as GAAP motivates (Stanley et al., 2026). Combining the two yields defense in depth: MemScope minimizes the data entering the agent; downstream IFC constrains where authorized data may flow. The composition still needs an explicit declassification boundary for useful responses and cannot ignore implicit/model-mediated flows (Sabelfeld and Myers, 2003).

12.3. Inference and aggregation

An item-level policy may allow several innocuous facts whose combination reveals a protected attribute. A derivation meet prevents a stored summary from being broader than its cited parents, but it does not stop the response model from making an inference transiently or an adversary from accumulating answers. DP-MemView's per-attribute ledgers illustrate a complementary approach (Kim et al., 2026). A future MemScope extension could meter views by sensitive attribute, but doing so changes both the formal model and utility objective and is not claimed here.

12.4. Purpose granularity

Purposes that are too broad collapse to ordinary access control; purposes that are too narrow cause repeated denials and user prompts. The evaluation's scope-width sweep should therefore be treated as a design study,

not merely a security benchmark. A hierarchical purpose vocabulary may allow a policy to name "scheduling" while a token selects "schedule-medical-appointment." Inheritance must be explicit: membership in a child purpose does not automatically grant siblings.

12.5. Deletion semantics

Logical expiry is enforceable at the service boundary; physical deletion is more complicated because flash translation layers, backups, audit replicas, and authorized external recipients may retain copies. MemScope reports distinct states—inaccessible, tombstoned, ciphertext deleted, and key destroyed—rather than calling all of them "forgotten." This terminology avoids borrowing Vanish's all-copies objective (Geambasu et al., 2009) without its cryptographic/distributed assumptions.

12.6. Deployment paths

The narrowest deployment is a privileged prototype used by one research agent. A platform deployment could expose MemScope as a Mainline-compatible service. Enterprise deployment can add administrator policies but must preserve user visibility and avoid converting memory governance into hidden surveillance. Application-only emulation is useful for experiments, yet it cannot claim OS-level complete mediation.

12.7. Novelty boundary after concurrent work

The 2026 literature materially narrows novelty. Audience-Bound Memory already demonstrates lifecycle authorization and restrictive derivation by audience (Liu, 2026); MNC already binds disclosures to purpose and lifetime (Xu et al., 2026); GAAP already enforces agent data policies outside the model (Stanley et al., 2026). The defensible MemScope contribution is their systems intersection with Android identities, per-document pre-ranking views, mobile lifecycle/eviction, and emulator/reference-model measurements. If a newer paper demonstrates that same combination before submission, the manuscript must reposition as an implementation/evaluation study rather than claim conceptual novelty.

13. Limitations and ethical considerations

Design-stage artifact.. The largest limitation is that this artifact is a design manuscript with a reference implementation and an emulator prototype

plan. All system, attack, utility, retention, and cost outcomes are pending. The paper is not submission-ready as an empirical systems paper until the prototype and preregistered experiments are completed.

Trusted computing base.. Compromise of the kernel, MemScope service, issuer, policy compiler, trusted clock, or key-handling path invalidates the core proof. Complete mediation is particularly fragile during migration because agents may retain private databases, caches, logs, or cloud histories.

Semantic policy specification.. The OS can authenticate an app or tool more directly than it can determine a user's purpose. AppFunction metadata and visible task state reduce ambiguity but do not solve it. Incorrect defaults can cause both leakage and denial. Policy usability requires separate study.

Authorized-recipient behavior.. Once a correctly authorized tool receives plaintext, MemScope cannot stop it from retaining or disclosing the data unless downstream mediation also applies. The theorem must not be cited as an end-to-end noninterference result.

Inference leakage.. Labels apply to items and derivations, not every semantic consequence of a transcript. Repeated permitted answers can reveal protected attributes. Differential-privacy or query-auditing mechanisms may be necessary for high-risk deployments (Kim et al., 2026; Dwork and Roth, 2014).

Attribution error.. Natural-language entailment and overlap checks can accept a laundering derivation or reject a useful abstraction. Restrictive labels limit where an accepted item flows but do not validate truth. High-risk classes should use structured/extractive transforms or human review.

Availability and side channels.. An attacker can cause quarantine, storage pressure, token churn, or expensive verification. Result counts, timing, storage occupancy, and power may reveal information. Rate limits and coarse errors help, but access-pattern privacy and denial-of-service resistance are not proven.

Emulator-only evaluation.. The prototype runs on the standard Android emulator and the typed Python reference model. It does not establish complete mediation, vendor secure-hardware behavior, or phone-level latency, energy, and thermal costs. AppSearch PlatformStorage, AppFunctions interception, StrongBox/Knox, and vendor services remain future work. AppSearch visibility is studied as an emulated baseline, not as a modified platform module.

Dataset realism and ethics.. Synthetic profiles reduce risk to real users but may not reproduce real disclosure norms, languages, disability needs, household structures, or adversarial behavior. No real personal memory should be collected without consent, minimization, and ethics approval. Attack artifacts must be isolated from production accounts and tools.

14. Conclusion

MemScope treats mobile-agent memory as an operating-system protection object rather than an application prompt. Each persistent item is bound to provenance, purpose, app/tool consumers, sensitivity, and lifetime; each read is authorized by a task-rooted token before semantic ranking; and each ordinary derivation inherits the meet of its sources' scopes. Retrieval-time expiry separates logical safety from delayed deletion, while policy-aware eviction makes resource pressure explicit without widening authorization.

Under authentic-token, complete-mediation, state-integrity, trusted-time, and ranker-confinement assumptions, the formal model establishes admission soundness, derived-label monotonicity, returned-plaintext cross-scope safety, and expiry safety. It does not establish unrestricted semantic privacy, truthfulness of generated memories, or control after authorized disclosure. Those boundaries are essential to the scientific claim.

The remaining work is empirical: implement the Android emulator prototype, audit the mediation boundary, measure the emulator and reference implementation, reproduce published extraction and poisoning attacks, quantify utility and attribution errors, and replace every placeholder with traceable results. Until that work is complete, MemScope is a rigorously specified system design, reference implementation, and evaluation plan, not evidence of achieved attack reduction or system performance.

Appendix A. Detailed proofs and model boundaries

Appendix A.1. Auxiliary definitions

For an execution trace π, let $\mathsf{release}(i, q, v)$ denote a MemScope API event returning plaintext value $v = x_i$ for item i under verified context q. Internal index hits, ciphertext reads, and authorization denials are not release events. Let $\mathsf{epoch}(q, t)$ be the applicable policy epoch at time t. A release is *snapshot-consistent* when all authorization fields and the final epoch check refer to one serializable state.

Define the coordinate order $\preceq$ precisely. For

$$\ell = (P, A, T, C, s, b, e), \qquad \ell' = (P', A', T', C', s', b', e'),$$

we have $\ell \preceq \ell'$ iff

$$P \subseteq P',\ A \subseteq A',\ T \subseteq T',\ C \subseteq C',\ s \succeq_{\mathbb{S}} s',\ b \geq b',\ e \leq e'.$$

Here $s \succeq_{\mathbb{S}} s'$ means that s requires controls at least as restrictive as s'. The order is reflexive and transitive because subset inclusion and the scalar orders are reflexive and transitive coordinatewise.

Appendix A.2. Retrieval admission soundness

Full proof of Theorem 1. Consider a successful release event $\mathsf{release}(i, q, x_i)$. By Algorithm 1, execution reaches decryption only after VerifyToken succeeds. Assumption 1 implies that q is the context signed by the trusted issuer and is bound to the actual caller and tool chain; the adversary has not substituted an app, tool, consumer, purpose, expiry, or policy epoch.

The index produces candidate set C. Its correctness is not required for safety: an under-approximation can hurt utility, and an over-approximation can increase work. For item i to enter list A, the trusted monitor explicitly evaluates $\mathsf{Auth}(m_i, q)$ and the integrity check. By Assumption 3, the checked label and provenance are the integrity-protected values associated with m_i.

By Assumption 4, ranking returns only identifiers contained in A; therefore $i \in A$. Before decryption, Algorithm 1 compares the current policy epoch and token time with the values validated for the query. If they changed, the algorithm restarts and the considered execution is not the successful release. Hence the release uses a snapshot in which $\mathsf{Auth}(m_i, q)$ held. Finally, `DecryptAuthorized` accepts only the already selected identifiers and checks their authenticated label/ciphertext binding. Thus successful plaintext release implies $\mathsf{Auth}(m_i, q)$ at the release snapshot.

The argument relies on complete mediation: without Assumption 2, a caller might obtain x_i from a shadow store without producing a MemScope release event. The theorem does not speak about that path. □

Appendix A.3. Derived-label safety over a DAG

Lemma 3 (Path monotonicity)**.** *If $v_0 \to v_1 \to \cdots \to v_r$ is a path of ordinary admitted derivations in $\mathcal{G}$, then $\ell_{v_r} \preceq \ell_{v_0}$.*

Proof. For each edge $v_{j-1} \to v_j$, Theorem 2 gives $\ell_{v_j} \preceq \ell_{v_{j-1}}$. Repeated transitivity proves the claim. □

Expanded proof of Theorem 2. An admitted call passed line 11 of Algorithm 2, so all parents were read in one snapshot and the proposed label satisfied $\ell_d \preceq \ell^*$, with $\ell^* = \mathsf{meet}(S)$. For each parent j, intersection gives $P^* \subseteq P_j$, $A^* \subseteq A_j$, $T^* \subseteq T_j$, and $C^* \subseteq C_j$; $b^* = \max b_j \geq b_j$; $e^* = \min e_j \leq e_j$; and $s^* = \bigvee s_j \succeq s_j$. Thus $\ell^* \preceq \ell_j$. Transitivity yields $\ell_d \preceq \ell_j$. This proof is independent of the semantic verifier's accuracy: the verifier is necessary for content integrity, but label non-widening follows from deterministic checks. □

Appendix A.4. Cross-scope returned-plaintext safety

Full proof of Theorem 3. First consider direct retrieval of m under q_2. If the service returned x_m, Theorem 1 would imply $\mathsf{Auth}(m, q_2)$, contradicting the premise $\neg\mathsf{Auth}(m, q_2)$. Hence no direct release occurs.

Now suppose a caller attempts to obtain the same stored content through an ordinary derived item d that cites m directly or transitively. Because $\mathcal{G}$ is acyclic and finite at every store snapshot, the ancestor relation has finite paths. Lemma 3 gives $\ell_d \preceq \ell_m$. The authorization predicate is monotone with respect to restrictiveness: if $\ell_d \preceq \ell_m$ and q_2 is unauthorized for m, then q_2 cannot become authorized for d through a set or time coordinate that was absent from m. Sensitivity adds controls but never grants access. Therefore $\neg\mathsf{Auth}(d, q_2)$ for the coordinate responsible for the original denial. Applying Theorem 1 again rules out release of x_d under q_2.

The theorem refers to stored plaintext and ordinary derivations. A separately authorized declassification is a new policy event and lies outside the premise. Likewise, a model might infer or paraphrase a related fact from other authorized items; that is not a release of m or a descendant governed by this proof. □

Appendix A.5. Expiry under races

Lemma 4 (Final-check freshness)**.** *If a token or item expires after candidate filtering but before final release, the release does not occur under the expired snapshot.*

Proof. Algorithm 1 checks token time and policy epoch before decryption. Item expiry is evaluated inside the release-snapshot authorization recheck in the implementation contract. A failed check restarts or denies. Trusted

monotonic time prevents rollback from making an expired interval current again. □

Expanded proof of Theorem 4. For all contexts q at trusted time $\tau \geq e_i$, the conjunct $\tau < e_i$ is false. Thus $\mathsf{Auth}(m_i, q)$ is false. Theorem 1 excludes a release. If expiry races with an in-flight query, the final-check lemma excludes release after expiry. Ciphertext may remain until compaction, but Assumption 2 denies callers a separate decryption path. □

Appendix A.6. Countermodels when assumptions fail

Forged or confused token.. If a shopping tool can present a token bound to a health tool, an item restricted only by the claimed token fields can be returned. Caller/tool-chain binding in Assumption 1 is therefore necessary.

Shadow memory.. If the application copies an authorized health item into a private unscoped vector store, a later shopping task may retrieve it without MemScope. This violates Assumption 2, not the authorization predicate.

Clock rollback.. If the clock can move from $\tau \geq e_i$ back to $\tau' < e_i$, read-time TTL alone may re-authorize the item. A trusted monotonic clock or securely persisted high-water mark is required.

Ranker with direct store access.. A reranker that fetches arbitrary payloads by ID can bypass the candidate view. Assumption 4 therefore requires process/API confinement, not merely an instruction to respect IDs.

Authorized semantic inference.. Let two authorized items state a person's recurring clinic location and appointment specialty. A model may infer a diagnosis not present as a stored item. All MemScope checks can succeed while the inferred attribute leaks. This is why the theorem is not semantic noninterference.

Appendix A.7. Complexity details

If allowed sets are stored as sorted arrays, a check costs logarithmic time in each set size; keyed hash sets give expected constant time with additional space. The $O(c)$ statement in Proposition 1 assumes the latter or bounded set cardinalities. A bitmap representation yields $O(1)$ membership and compact storage for small static vocabularies. Token signature verification is $O(1)$ in the number of memory items but may dominate small queries; caching verified token digests changes constants, not the asymptotic bound.

Compaction's greedy sort costs $O(n \log n)$, or $O(n)$ expected time with bucketed priorities. Exact 0–1 knapsack is pseudo-polynomial in budget and inappropriate for the online path; it serves only as an offline oracle for small evaluation instances. No approximation ratio is claimed for the multi-factor heuristic.

Appendix B. Detailed experimental protocol

Appendix B.1. Episode construction

Each synthetic person is assigned 30–180 days of events. Sources include messaging, calendar, health, shopping, travel, and system settings. Every atomic fact has a stable ID, source timestamp, source app/tool, sensitivity, lifetime, authorized purpose/consumer set, and update chain. Natural-language renderers produce multiple paraphrases while preserving the structured oracle. A held-out auditor verifies referential consistency and policy labels before an episode enters the benchmark.

Tasks are sampled in four groups: single-app recall; authorized cross-app composition; unauthorized cross-purpose probes; and temporal update/expiry. Compound tasks are decomposed into token sequences, enabling measurement of whether agents request unnecessarily broad views. The same structured episode generates benign and adversarial variants so paired comparisons differ only in attack content.

Appendix B.2. Attack adaptation rules

For every published attack, the artifact record states original threat model, attacker access, target memory architecture, attack budget, and success definition. Adaptation to MemScope may change only the connector necessary to call the same logical agent interface. The attack does not receive extra knowledge of labels unless that knowledge is part of an explicit white-box track. Results are reported as reproduction, partial reproduction, or reimplementation.

MEXTRA and ADAM run from a query-only consumer scope. SPORE registers a tool with one legitimate purpose and attempts to extract items outside that purpose; a second within-scope condition tests the admitted limitation. MINJA and MemPoison submit candidate content through normal interactions; success is separated into record admission, later retrieval, and behavioral effect. An always-allow store establishes attack viability, and a

no-memory agent checks whether the measured disclosure truly comes from memory.

Appendix B.3. Policy annotation reliability

Two annotators independently label a stratified subset of purpose, consumer, lifetime, and sensitivity dimensions using a written handbook. Agreement is reported per dimension with raw agreement and an appropriate chance-corrected statistic. Disagreements are adjudicated without inspecting system outputs. The remaining synthetic corpus can inherit deterministic labels from the generator, but natural-language tasks still receive a leakage audit.

Appendix B.4. Microbenchmarks

Microbenchmarks measure cold and warm token verification, exact label checks, indexed projections, encryption/decryption by payload size, audit append, derivation meet by parent count, attribution-verifier latency, tombstone writes, compaction, and key rotation. Store sizes grow geometrically. Measurements distinguish cache-cold reboot, process-cold, and steady-state conditions.

The MobiMem-reported retrieval latency is contextual evidence, not a directly comparable baseline unless code, device, model, index size, and metric definition are reproduced (Liu et al., 2025). MemScope will report its own distributions and only compare under a matched harness.

Appendix B.5. End-to-end tasks

End-to-end experiments replay full agent trajectories with identical model prompts except for the memory adapter. The harness records token requests, authorized candidate counts, selected items, tool arguments, final answer, policy violations, latency breakdown, host resource window, and model usage. A blinded evaluator scores answer correctness where deterministic metrics are unavailable. Tool arguments are checked atom-by-atom against purpose-specific need-to-know policy, following ToolPrivacyBench's trajectory-level motivation (Hu et al., 2026).

Appendix B.6. Eviction experiment

At fixed byte budgets (5, 10, 20, 40, 80, and 100% of uncompressed store size), compare least-recently used, semantic-relevance, learned importance, safety-first, sensitivity-delete-first, and MemScope's policy-aware heuristic.

Future questions remain hidden at eviction time. Outcomes are entitled QA accuracy, safety-critical survival, invalidated-fact retention, sensitive byte-days, and compaction cost. Exact knapsack on small instances estimates the heuristic gap without claiming that the objective perfectly represents user welfare.

Appendix B.7. Derivation corpus

Construct source groups with: faithful extractive summaries; faithful abstractions; unsupported additions; contradictions; cross-scope combinations; omitted sensitive qualifiers; expired parents; stale parent versions; and empty label intersections. Human annotation identifies support at claim/span level. Compare lexical overlap, small local NLI, larger local/cloud NLI, and structured-only transforms. Report false acceptance separately for high-sensitivity and ordinary classes.

Appendix B.8. Planned experiment matrix

Table B.5: Planned evaluation matrix. Every cell is pending implementation.

Question	Independent variables	Primary metrics	Required controls
RQ1 enforcement	attack, baseline, scope width, tool trust	violations, extraction yield, attack cost	always-allow, no-memory, matched ranker/model
RQ2 utility	benchmark, scope width, k, token decomposition	entitled recall, answer accuracy, abstention	policy-equivalent oracle, unscoped ceiling
RQ3 derivation	verifier, transform, sensitivity, parents	false widening, attribution FAR/FRR, quarantine	deterministic meet, always-accept/reject
RQ4 lifecycle	TTL, revocation timing, reboot, race	post-expiry releases, stale reuse, deletion lag	clock rollback and shadow-cache negatives

Question	Independent variables	Primary metrics	Required controls
RQ5 system cost	environment, store size, verifier, cold/warm	p50/p95/p99 latency, storage	identical emulator image, host metadata, software Keystore
RQ6 retention	budget, policy mix, eviction method	safety survival, FAMA, byte-days, compaction cost	future-blind queries, exact small oracle
RQ7 GUI tasks	environment, task group, memory policy	task success, tool-call count, memory-induced failures	matched app setup, flaky-task stratum

Appendix B.9. Result-release policy

The final paper will not replace "pending" with a single best run. A machine-readable manifest links each table cell and plot series to raw trace hashes and analysis code. Failed hypotheses remain reported. Any change to primary metrics, exclusion rules, attacks, or baselines after seeing results is disclosed as exploratory.

Appendix C. Policy and trace examples

Appendix C.1. Scheduling example

Suppose a user asks a messaging agent to propose times for a medical appointment. Calendar item m_1 contains free/busy intervals and permits purpose `scheduling`, apps {assistant, calendar}, tools {calendar.read}, and a seven-day lifetime. Health item m_2 contains a clinic preference and permits purposes {health, scheduling}, apps {assistant, health}, tools {health.preference.read}, and a 90-day lifetime. No single tool is authorized for both items. The issuer decomposes the task: one token retrieves free/busy intervals, another retrieves the clinic preference, and the agent receives purpose-limited views. A derived proposed-time item citing both sources has app intersection {assistant}, purpose intersection {scheduling}, tool intersection empty, and expiry no later than seven days. It can be shown

to the user by the assistant but is quarantined from generic tool retrieval unless policy explicitly names a common tool.

Appendix C.2. Shopping versus health

An allergy item allows purposes {health, food-safety} and tools {health.read, restaurant.check}. A shopping recommender asks under purpose `personalized-ads`; authorization fails even when the query embedding is close to the allergy text. Because filtering precedes ranking, the item neither appears in candidates nor displaces entitled shopping preferences. If the user explicitly asks the shopping agent to screen groceries for allergens, the issuer selects `food-safety` and the approved grocery checker tool; the same app can then receive the item under a different, visible task context.

Appendix C.3. Malicious tool persistence

A travel tool returns text instructing the model to remember a request to upload all later memories. The direct observation is labeled with source tool `travel.badtool`, travel purpose, short lifetime, and high integrity risk. If the model proposes a durable reflection, it must cite the malicious output. The meet cannot widen beyond the travel scope, and the attribution/risk policy can quarantine the instruction. Even if admitted, a later health token cannot retrieve it. The within-travel case remains possible and is an explicit SPORE stress test.

Appendix C.4. Correction and invalidation

At time t_1 an item says "preferred pharmacy is A." At t_2 the user changes it to B. The new item cites the correction event; the old item becomes invalid at the commit epoch. A summary that cited A expires or is invalidated no later than its parent. Memora-style queries test that B is returned and A is not reused (Uddin et al., 2026). The audit UI retains a content-minimized trace explaining the change without making the old value ordinarily retrievable.

Appendix C.5. Illustrative policy fragment

```
rule food_safety_allergy {
  source_classes = [health_record, user_statement]
  purposes       = [health, food_safety]
  consumers      = [primary_assistant]
  apps           = [assistant_pkg, health_pkg]
  tools          = [health.read, restaurant.check,
```

```
                    grocery.allergen_check]
  sensitivity    = health_sensitive
  max_ttl_days   = 365
  derivation     = extractive_or_confirmed
  retention      = safety_pinned_until_expiry
}
```

The syntax is illustrative, not an implemented configuration language. The actual policy compiler must define canonical IDs, inheritance, conflict resolution, and user-facing explanations.

Appendix D. Artifact and reproducibility requirements

Appendix D.1. Minimum artifact

The minimum credible artifact contains source and build instructions for the reference implementation and the Android emulator prototype; the emulator image and API level; unit and integration tests; a reproducible demonstration build; policy examples; the synthetic workload generator; attack adapters; benchmark preparation scripts; and analysis notebooks/scripts. Binary-only release is insufficient for evaluating label propagation and retrieval mediation.

Appendix D.2. Security test suite

The suite mutates each token field; replays nonces; swaps package signatures; inserts an unregistered tool; nests a broader child token; races expiry and revocation; rolls the wall clock backward; corrupts labels and provenance; references missing or stale parents; proposes broadened derived labels; bypasses the index projection; forces ranker exceptions; fills the audit partition; exhausts storage with pinned data; reboots during compaction; and attempts a shadow-store bypass. Each test states the expected denial, quarantine, restart, or explicit failure.

Appendix D.3. Deterministic formal-model tests

Property-based tests generate finite policy lattices, item labels, contexts, and derivation DAGs. They check: admitted items satisfy authorization; meet is below every operand; ordinary derivation never widens any coordinate; expired items are never returned; revocation epochs prevent stale releases; and authorization remains independent of ranking order. Counterexample shrinking produces minimal failing traces.

Appendix D.4. Documentation checklist

- Exact compiler, JDK, Android SDK/NDK, kernel, AppSearch module, and build flags.
- Exact devices, firmware/build fingerprints, battery condition, and secure-hardware features.
- Model provider/version/date, local model hashes, quantization, prompts, and decoding parameters.
- Dataset versions, licenses, checksums, transformations, exclusions, and policy annotations.
- Attack commit hashes, reproduction status, deviations, and success definitions.
- Raw traces with sensitive/synthetic fields documented; analysis environment and random seeds.
- Figure/table generation commands and a script checking that every plotted point has a source record.
- A limitations file mapping every formal assumption to one or more implementation tests.

Appendix D.5. Ethical release

Attack adapters should default to synthetic local endpoints, require explicit configuration for any network sink, and avoid credentials. Synthetic person records remain clearly labeled as synthetic. If user-study or real-device personal data is later collected, the artifact must exclude it by default and document consent, retention, de-identification, and access controls.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this manuscript.

Data and code availability

The current artifact contains a design, formal model, typed reference implementation, Android emulator prototype plan, and preregistered evaluation protocol. No completed experimental measurements are claimed. Implementation, workload generators, policies, raw traces, and analysis scripts must be released or archived before submission; see the reproducibility appendix and `TODO_before_submission.md`.

References

Agrawal, R., Kiernan, J., Srikant, R., Xu, Y., 2002. Hippocratic databases, in: Proceedings of the 28th International Conference on Very Large Databases, pp. 143–154. URL: https://www.vldb.org/conf/2002/S05P02.pdf, doi:10.1016/B978-155860869-6/50021-4.

Android Developers, 2026a. AppFunctions api reference. Official Android API reference. URL: https://developer.android.com/reference/androidx/appfunctions/AppFunctionSchemaDefinition. accessed 2026-10-03.

Android Developers, 2026b. AppSearch overview. Official Android documentation. URL: https://developer.android.com/develop/ui/views/search/appsearch. accessed 2026-10-03.

Android Developers, 2026c. SetSchemaRequest.Builder: Schema visibility for packages. Official Android API reference. URL: https://developer.android.com/reference/androidx/appsearch/app/SetSchemaRequest.Builder. accessed 2026-10-03.

Android Open Source Project, 2026. Hardware-backed keystore and StrongBox. Official Android security documentation. URL: https://source.android.com/docs/security/features/keystore. accessed 2026-10-03.

Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Traon, Y.L., Octeau, D., McDaniel, P., 2014. FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps, in: Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 259–269. doi:10.1145/2594291.2594299.

Chen, Y., Zhao, J., Tang, B., Wang, H., Xiong, F., Li, Z., 2026. MemPrivacy: Privacy-preserving personalized memory management for edge-cloud agents. URL: https://arxiv.org/abs/2605.09530, arXiv:2605.09530.

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F., 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. URL: https://arxiv.org/abs/2406.13352, arXiv:2406.13352.

Denning, D.E., 1976. A lattice model of secure information flow. Communications of the ACM 19, 236–243. doi:10.1145/360051.360056.

Dong, S., Xu, S., He, P., Li, Y., Tang, J., Liu, T., Liu, H., Xiang, Z., 2025. A practical memory injection attack against LLM agents. URL: https://arxiv.org/abs/2503.03704, arXiv:2503.03704.

Dwork, C., Roth, A., 2014. The Algorithmic Foundations of Differential Privacy. volume 9 of *Foundations and Trends in Theoretical Computer Science*. Now Publishers. doi:10.1561/0400000042.

Enck, W., Gilbert, P., Chun, B.G., Cox, L.P., Jung, J., McDaniel, P., Sheth, A.N., 2010. TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones, in: 9th USENIX Symposium on Operating Systems Design and Implementation, USENIX Association. pp. 393–407. URL: https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Enck.pdf.

Gao, J., Xia, K., Zhang, Y., Hong, X., Lin, M., Wei, X., Li, W., Lu, S., 2026a. MemPoison: Uncovering persistent memory threats and structural blind spots in LLM agents. URL: https://arxiv.org/abs/2607.14651, arXiv:2607.14651.

Gao, X., Chen, W., Meng, X., Wang, L., Zang, C., Wang, J., Li, Z., Guo, S., 2026b. Isolated but exposed: Persistence-based memory extraction attack on LLM agents. URL: https://arxiv.org/abs/2607.23444, arXiv:2607.23444.

Geambasu, R., Kohno, T., Levy, A.A., Levy, H.M., 2009. Vanish: Increasing data privacy with self-destructing data, in: 18th USENIX Security Symposium, USENIX Association. pp. 299–315. URL: https://www.

usenix.org/conference/usenixsecurity09/technical-sessions/presentation/vanish-increasing-data-privacy-self.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. URL: https://arxiv.org/abs/2302.12173, arXiv:2302.12173.

Gupta, S., Sepahvand, N.M., Kumar, A., Subakan, C., Gella, S., Noël, P.A., Taslakian, P., Bagdasarian, E., Zantedeschi, V., 2026. PiSAs: Benchmarking contextual integrity in multi-user agentic systems. URL: https://arxiv.org/abs/2607.05318, arXiv:2607.05318.

Hu, S., Liu, L., Meng, Z., Zhao, Z., 2026. ToolPrivacyBench: Benchmarking purpose-bound privacy in tool-using LLM agents. URL: https://arxiv.org/abs/2606.28061, arXiv:2606.28061.

Kim, J.W., Min, B., Edemacu, K., Choi, Y., Cho, S.H., Jang, B., 2026. DP-MemView: A memory interface for attribute-level transcript privacy in long-term LLM agents. URL: https://arxiv.org/abs/2608.03130, arXiv:2608.03130.

Lam, C., Li, J., Zhang, L., Zhao, K., 2026. Governing evolving memory in LLM agents: Risks, mechanisms, and the stability and safety governed memory (SSGM) framework. URL: https://arxiv.org/abs/2603.11768, arXiv:2603.11768.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., Kiela, D., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, pp. 9459–9474. URL: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

Li, H., 2025. Memory as a service (MaaS): Rethinking contextual memory as service-oriented modules for collaborative agents. URL: https://arxiv.org/abs/2506.22815, arXiv:2506.22815.

Liu, S., 2026. Audience-bound persistent memory: Authorization across the memory lifecycle. URL: https://arxiv.org/abs/2609.36373, arXiv:2609.36373.

Liu, X., Teng, H., Li, C., Ni, Y., Lu, S., Wang, A., Liu, Y., Wang, Y., Li, X., Yi, X., Li, Y., Zhao, K., Liang, J., Chen, Y., Chen, J., Qu, H., Shao, K., Luan, J., 2026. Mi-Memory: A lifecycle memory framework for personal AI. URL: https://arxiv.org/abs/2607.18975, arXiv:2607.18975.

Liu, Z., Zhang, C., Zhao, X., Feng, Y., Bai, B., Feng, D., Feng, E., Xia, Y., Chen, H., 2025. Beyond training: Enabling self-evolution of agents with MOBIMEM. URL: https://arxiv.org/abs/2512.15784, arXiv:2512.15784.

Lyu, X., He, J., Wang, N., Hu, Y., Li, T., Chen, D., Li, S., Chen, Y., 2026. ADAM: A systematic data extraction attack on agent memory via adaptive querying. URL: https://arxiv.org/abs/2604.09747, arXiv:2604.09747.

Maharana, A., Lee, D.H., Tulyakov, S., Bansal, M., Barbieri, F., Fang, Y., 2024. Evaluating very long-term conversational memory of LLM agents, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. URL: https://arxiv.org/abs/2402.17753.

Myers, A.C., Liskov, B., 1998. Complete, safe information flow with decentralized labels, in: Proceedings of the 1998 IEEE Symposium on Security and Privacy, pp. 186–197. doi:10.1109/SECPRI.1998.674834.

Myers, A.C., Liskov, B., 2000. Protecting privacy using the decentralized label model. ACM Transactions on Software Engineering and Methodology 9, 410–442. doi:10.1145/363516.363526.

Nissenbaum, H., 2010. Privacy in Context: Technology, Policy, and the Integrity of Social Life. Stanford University Press.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E., 2023. MemGPT: Towards LLMs as operating systems. URL: https://arxiv.org/abs/2310.08560, arXiv:2310.08560.

Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S., 2023. Generative agents: Interactive simulacra of human behavior, in: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. URL: https://doi.org/10.1145/3586183.3606763, doi:10.1145/3586183.3606763.

Rawles, C., Li, A., Rodriguez, D., Riva, O., Lillicrap, T., 2024. AndroidWorld: A dynamic benchmarking environment for autonomous agents. URL: https://arxiv.org/abs/2405.14573, arXiv:2405.14573.

Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C.J., Hashimoto, T., 2023. ToolEmu: Identifying the risks of LM agents with an LM-emulated sandbox. URL: https://arxiv.org/abs/2309.15817, arXiv:2309.15817.

Sabelfeld, A., Myers, A.C., 2003. Language-based information-flow security. IEEE Journal on Selected Areas in Communications 21, 5–19. doi:10.1109/JSAC.2002.806121.

Saltzer, J.H., Schroeder, M.D., 1975. The protection of information in computer systems. Proceedings of the IEEE 63, 1278–1308. doi:10.1109/PROC.1975.9939.

Selvam, S., Ghosh, A., 2026. ProfileFoundry: A synthetic person-object substrate for privacy, memory, and tool-use evaluation in LLM agent. URL: https://arxiv.org/abs/2606.26403, arXiv:2606.26403.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K.R., Yao, S., 2023. Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems. URL: https://arxiv.org/abs/2303.11366.

Stanley, R., Verma, A., Tsai, L., Kallas, K., Kumar, S., 2026. An AI agent execution environment to safeguard user data. URL: https://arxiv.org/abs/2604.19657, arXiv:2604.19657.

Uddin, M.N., Shubham, K., Blanco, E., Baral, C., Wang, G., 2026. From recall to forgetting: Benchmarking long-term memory for personalized agents. URL: https://arxiv.org/abs/2604.20006, arXiv:2604.20006.

Wang, B., He, W., Zeng, S., Xiang, Z., Xing, Y., Tang, J., He, P., 2025. Unveiling privacy risks in LLM agent memory, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics. pp. 25241–25260. URL: https://aclanthology.org/2025.acl-long.1227/, doi:10.18653/v1/2025.acl-long.1227.

Wang, F., Ko, R., Mickens, J., 2019. Riverbed: Enforcing user-defined privacy constraints in distributed web services, in: 16th USENIX Symposium on Networked Systems Design and Implementation, USENIX Association. pp. 615–630. URL: https://www.usenix.org/conference/nsdi19/presentation/wang-frank.

Wang, H., Yang, S., Chen, Y., Liu, P., 2026. Hijacking agent memory: Stealthy trojan attacks through conversational interaction. URL: https://arxiv.org/abs/2605.29960, arXiv:2605.29960.

Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.W., Yu, D., 2025. LongMemEval: Benchmarking chat assistants on long-term interactive memory, in: International Conference on Learning Representations. URL: https://arxiv.org/abs/2410.10813, arXiv:2410.10813.

Xie, W., Guo, S., Zhang, F., Xia, T., Yang, X., Ma, L., Yan, J., Ren, Q., 2026. MemEvoBench: Benchmarking memory misevolution in LLM agents. URL: https://arxiv.org/abs/2604.15774, arXiv:2604.15774.

Xu, J., Fan, L., Wang, Z., Li, X., Liu, H., 2026. MNC: Scope-bound semantic declassification for private LLM-agent communication. URL: https://arxiv.org/abs/2608.01719, arXiv:2608.01719.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y., 2023. ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations. URL: https://arxiv.org/abs/2210.03629.

Zhang, L., Yang, H., Ouyang, X., 2026. Benchmarking memory architectures for mobile agents in short-term and long-term applications, in: Proceedings of the 24th Annual International Conference on Mobile Systems, Applications and Services Workshops, pp. 175–181. URL: https://doi.org/10.1145/3812836.3814752, doi:10.1145/3812836.3814752.

Zhang, S., Dinan, E., Urbanek, J., Szlam, A., Kiela, D., Weston, J., 2018. Personalizing dialogue agents: I have a dog, do you have pets too?, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, pp. 2204–2213. URL: https://aclanthology.org/P18-1205/, doi:10.18653/v1/P18-1205.

Zhong, W., Guo, L., Gao, Q., Ye, H., Wang, Y., 2024. MemoryBank: Enhancing large language models with long-term memory, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 19724–19731. URL: https://ojs.aaai.org/index.php/AAAI/article/view/29946.

Zhu, L., Lin, Y., Wang, Y., Liu, Y., Li, H., Chen, Y., 2026. Will my assistant remember my allergy? what personal LLM assistants forget when conversation memory is compressed, in: Proceedings of the ACM HumanSys Workshop. URL: https://arxiv.org/abs/2609.05767, doi:10.1145/3842436.3843817.

Algorithm 2 MemScope derived-memory admission

Require: token κ, payload x_d, parent IDs S, proposed label $\hat{\ell}$, transformation f

$q \leftarrow \mathsf{VerifyToken}(\kappa, \mathsf{BinderCaller}, \mathsf{ToolChain})$
if $q = \bot$ or $S = \emptyset$ **then**
    return REJECT
end if
$M_S \leftarrow \mathsf{ReadParentsAtomically}(S)$
if $|M_S| \neq |S|$ **then**
    return REJECT
end if
for all $m_j \in M_S$ **do**
    if $\neg\mathsf{Auth}(m_j, q)$ **then**
        return REJECT
    end if
end for
$\ell^* \leftarrow \mathsf{meet}(M_S)$
if $\neg(\hat{\ell} \preceq \ell^*)$ **then**
    return REJECT
end if
if $\mathsf{EmptyRequiredScope}(\ell^*)$ **then**
    return QUARANTINE
end if
if $\neg\mathsf{Attributed}(x_d, M_S, f)$ **then**
    return QUARANTINE
end if
$z_d \leftarrow \mathsf{DerivationProvenance}(S, f, q)$
return $\mathsf{Commit}(x_d, z_d, \hat{\ell})$

Algorithm 3 MemScope policy-aware compaction

Require: store $\mathcal{M}$, byte budget B, trusted time τ

delete/tombstone every m_i with $\tau \geq e_i$ or explicit revocation
rotate or destroy keys for empty revoked compartments
if $\mathsf{Bytes}(\mathcal{M}) \leq B$ **then**
    return
end if
$H \leftarrow \mathsf{HardPinned}(\mathcal{M})$
if $\mathsf{Bytes}(H) > B$ **then**
    return RESOURCE_CONFLICT
end if
for all $m_i \in \mathcal{M} \setminus H$ **do**
    $\rho_i \leftarrow (\alpha u_i + \beta r_i + \gamma c_i - \delta d_i - \eta g_i)/w_i$
end for
for all m_i in nondecreasing ρ_i **do**
    preserve required lineage metadata; evict m_i
    if $\mathsf{Bytes}(\mathcal{M}) \leq B$ **then**
        return
    end if
end for